TARGET SPEECH EXTRACTION ON EDGE DEVICE

A project report submitted in partial fulfilment
of the requirements for the degree of

Bachelor of Technology
in
Electronics and Computer Engineering
by

Johaan Sam Sanil (23BLC1286)
Rinol Jeo Manoj (23BLC1207)
Navneeth Regikumar (23BLC1276)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Electronics Engineering
Vellore Institute of Technology, Chennai
Vandalur - Kelambakkam Road
Chennai – 600127, India

October 2026



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Electronics Engineering

Declaration

We hereby declare that the report titled **Target Speech Extraction on Edge Device** submitted by us to the School of Electronics Engineering, Vellore Institute of Technology, Chennai, in partial fulfilment of the requirements for the award of Bachelor of Technology in Electronics and Computer Engineering, is a bonafide record of the work carried out by us under the supervision of A Bharathi Sankar.

We further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or of any other institute or University.

Signature of the Student:_……

Name and Reg. No.:_…….

Date: ……

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Electronics Engineering

Certificate

This is to certify that the project report titled **Target Speech Extraction on Edge Device** submitted by Johaan Sam Sanil (23BLC1286), Rinol Jeo Manoj (23BLC1207) and Navneeth Regikumar (23BLC1276) to Vellore Institute of Technology, Chennai, in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology in Electronics and Computer Engineering is a bonafide work carried out under my supervision. The project report fulfils the requirements as per the regulations of this University and, in my opinion, meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Supervisor	**Head of the Department**
Signature: ……	Signature: ……
Name: A Bharathi Sankar	Name: ……
Date: ……	Date: ……

Examiner 1	**Examiner 2**
Signature: ……	Signature: ……
Name: ……	Name: ……
Date: ……	Date: ……

(Seal of the School)

Abstract

A hearing aid in a busy canteen, an earbud on a video call and a smart speaker in a crowded room all face the same problem: two, three or four people talk at once and only one of them matters. Target speech extraction (TSE) solves it with a few seconds of the wanted speaker recorded alone, which tells the network whose voice to keep. For such products the model has to run on the device and cope with more than two talkers. Published TSE work, however, mostly reports two-speaker results on server GPUs.

We designed, trained and deployed a time-domain TSE network for this setting. It pairs a U-shaped convolutional separator with speaker conditioning by feature-wise linear modulation and with speaker-driven gates that split and merge features across stages. We trained a single model on mixtures of two to four LibriSpeech speakers, generated afresh in every epoch. On Libri2Mix, Libri3Mix and our four-speaker extension it improves the scale-invariant signal-to-distortion ratio by 9.49, 8.13 and 7.00 dB. The same design trained on pairs only reaches 4.84 and 3.13 dB with three and four talkers. For deployment we split the network into an embedder, run once per speaker, and an extractor that runs on every block of audio. After ONNX export and 8-bit quantisation the model shrinks from 51 MB to 14 MB within 0.13 dB of the full-precision model. Our offline Android app ran it on three phones. The fastest processed speech 13 times faster than real time, every phone matched the PC quality, and live streaming worked with 1 s of delay (0.4 s for the compact variant, on a single CPU core). We also tried multi-head and differential attention in the separator bottleneck. Accurate extraction of one voice from up to four talkers fits comfortably within the memory and compute of a current smartphone.

Keywords: target speech extraction, speaker conditioning, multi-speaker mixtures, edge deployment, INT8 quantisation, ONNX, streaming inference.

Acknowledgements

We wish to express our sincere thanks and deep sense of gratitude to our project guide, A Bharathi Sankar, School of Electronics Engineering (SENSE), for the consistent encouragement and valuable guidance offered to us throughout the course of the project work.

We sincerely express our gratitude to **Dr. Ravi Sankar A**, Dean, SENSE; **Dr. Mohanaprasad K**, Associate Dean (Academics), SENSE; and **Dr. Chitra K**, Associate Dean (Research), SENSE, for providing the necessary facilities and for their constant encouragement and support throughout our project.

We also extend our heartfelt thanks to **Dr. Vijayakumar P**, Head of the Department, for his continuous guidance and support during the course of this project.

We also take this opportunity to thank all the faculty of the School for their support and their wisdom imparted to us throughout the course.

We thank our parents, family, and friends for bearing with us throughout the course of our project and for the opportunity they provided us in undergoing this course in such a prestigious institution.

Codes and Standards

Project Title: **Target Speech Extraction on Edge Device**

1. Speech Quality and Telephony Standards

ITU-T P.862 - Perceptual Evaluation of Speech Quality (PESQ)
Objective speech-quality measure used to score the extracted speech (narrow-band mode, 8 kHz).

ITU-T P.862.1 - Mapping function for PESQ to MOS-LQO
Maps raw PESQ scores to the mean-opinion-score scale reported in this work.

ITU-T G.711 - Pulse Code Modulation of voice frequencies
Defines the 8 kHz narrow-band telephony bandwidth used for all audio in this project.

2. Data Formats and Model Interchange

ONNX (Open Neural Network Exchange), operator set 17
Open standard used to export the trained network to a hardware-independent edge runtime.

IEEE 754-2019 - Floating-point arithmetic
Number format of the FP32/FP16 models; INT8 quantisation is defined relative to it.

RIFF/WAVE PCM audio format
Container format of all speech recordings, mixtures and model outputs.

3. Software Quality and Testing

ISO/IEC 25010 - Systems and software quality model
Guides the functional suitability, performance efficiency and maintainability goals of the software.

ISO/IEC/IEEE 29119 - Software testing
The pipeline is covered by automated unit and integration tests (shapes, metrics, checkpoints, streaming).

4. Data Protection and Security

Digital Personal Data Protection (DPDP) Act, 2023 (India)
Voice recordings are personal data; on-device processing keeps speech and enrollment data on the device.

ISO/IEC 27001 - Information security management
Applies to storage of the speaker enrollment embedding on the device.

5. Hardware Safety (edge device)

IEC 62368-1 - Audio/video, information and communication technology equipment safety
Safety requirements for the battery-powered or mains-powered audio device hosting the model.

Android Compatibility Definition / Google Play app policies
The application uses only public Android APIs and requests only the microphone permission.

Mapping of Sustainable Development Goals (SDGs)

Project Title: **Target Speech Extraction on Edge Device**

Mapped SDG	Justification	Example Impact
SDG 3 - Good Health and Well-being	Extracting the voice a listener wants to hear is the core function of a hearing aid in a noisy, multi-talker place.	Better speech understanding for people with hearing loss in restaurants, classrooms and family gatherings.
SDG 10 - Reduced Inequalities	On-device processing works offline on low-cost hardware, without paid cloud services.	Assistive listening becomes affordable and available where network access is poor.
SDG 4 - Quality Education	Isolating the teacher's voice from classroom babble improves lecture audio and captions.	Clearer recorded and live lectures for students, including those with hearing or attention difficulties.
SDG 9 - Industry, Innovation and Infrastructure	Compact, quantised neural speech processing that runs on edge processors.	A reusable, open pipeline (training, export, quantisation, streaming) for embedded speech products.

Table of Contents

Declaration i
Certificate ii
Abstract iii
Acknowledgements iv
Codes and Standards v
Mapping of Sustainable Development Goals (SDGs) vi
List of Figures ix
List of Tables x
List of Acronyms xi
1 Introduction 1
1.1 Background 1
1.2 Motivation for On-Device Processing 1
1.3 Problem Statement 2
1.4 Objectives 2
1.5 Contributions 3
1.6 Organisation of the Report 3
2 Literature Survey 4
2.1 Review of Prior Work 4
2.1.1 Blind Speech Separation 4
2.1.2 Target Speech Extraction 4
2.1.3 Speaker Conditioning and Attention 4
2.1.4 Speaker Embeddings 5
2.1.5 Datasets and Evaluation 5
2.1.6 Efficient On-Device Inference 5
2.2 Gaps Identified 5
3 Methodology 7
3.1 System Architecture 7
3.1.1 Architecture Overview 7
3.1.2 Encoder and Decoder 8
3.1.3 Speaker Encoder 8
3.1.4 U-ConvBlock Separator with FiLM Conditioning 8
3.1.5 Gated Embedding Split and Multi-stage Gated Feature Aggregation 9
3.1.6 Bottleneck Attention: Squeeze-and-Excitation, Multi-Head and Differential 9
3.2 Dataset and Data Pipeline 10
3.2.1 Datasets 10

3.2.2 Enrollment Protocol 11
3.2.3 Dynamic Multi-Speaker Mixing 11
3.3 Training Objective and Procedure 11
3.4 Evaluation Metrics 12
3.5 Edge Deployment Pipeline 13
3.5.1 Graph Split and ONNX Export 13
3.5.2 Quantisation 14
3.5.3 Streaming Inference 14
3.5.4 Target Edge Platform: Android Smartphone Application 14
3.6 Software and Hardware Setup 16
4 Results and Discussion 17
4.1 Results and Comparison 17
4.1.1 Extraction Quality versus Number of Speakers 17
4.1.2 Perceptual Quality, Interference and Artifacts 18
4.1.3 Training Behaviour 19
4.1.4 Error Analysis 19
4.1.5 Qualitative Example 20
4.2 Attention Study: Multi-Head and Differential Attention 21
4.3 Edge Deployment Results 22
4.3.1 Model Size and Complexity 22
4.3.2 Effect of Quantisation on Quality 23
4.3.3 Measurements on the Smartphone 24
4.3.4 Real-Time Streaming on the Smartphone 25
4.3.5 Results Across Three Smartphones 25
4.3.6 Streaming Latency versus Quality (Full Test Subsets) 26
4.4 Comparison of Existing Systems and the Proposed System 27
4.5 Negative Results 28
4.6 Limitations 28
5 Conclusion and Future Work 30
5.1 Conclusion 30
5.2 Future Work 30
Appendix 32
Bibliography 37
Biodata 40

List of Figures

1.1 Target speech extraction: a short enrollment of speaker A selects which voice is extracted from a single-microphone mixture of several talkers 1

3.1 Overall architecture of the proposed U-eConv-TSE network (R = 6 stages) 7

3.2 One U-ConvBlock separator stage with FiLM conditioning and bottleneck attention 9

3.3 Training data pipeline with dynamic 2-4 speaker mixing 11

3.4 Edge deployment pipeline: from the trained checkpoint to streaming inference on the device 13

3.5 Sliding-window streaming: only the newest hop of each 3 s window is output 14

3.6 The TSE Edge Android application: demo with a 3-speaker test mixture (left), extraction of the user's own voice (centre) and on-device benchmark (right) 16

4.1 Mean SI-SDRi on the test sets as the number of speakers grows 17

4.2 Validation SI-SDRi of the proposed model during training, for each number of speakers 19

4.3 Cumulative distribution of per-utterance SI-SDRi of the proposed model 19

4.4 Spectrograms of a three-speaker test mixture, the target reference and the extracted speech (an utterance at the median SI-SDRi of the test set) 20

4.5 Validation SI-SDRi per epoch for SE, MHSA and DIFF bottleneck attention (identical recipe; first 15 epochs) 21

4.6 SI-SDRi of the FP32 and INT8 versions on 2/3/4-speaker test subsets 23

4.7 Streaming quality of the proposed model against algorithmic latency; dashed lines: whole-utterance (offline) processing 27

List of Tables

2.1 Comparison of representative separation and extraction systems 5
3.1 Datasets used in this project 10
3.2 Model and training hyper-parameters 12
3.3 Development environment 16
4.1 Extraction quality on the 2-, 3- and 4-speaker test sets (higher SI-SDRi / SDRi is better; lower NSR is better) 17
4.2 Signal and perceptual metrics on the test sets (mixture → output for PESQ and STOI) 18
4.3 SI-SDRi (dB) of the proposed model by speaker gender (same gender: at least one interferer has the target's gender) 20
4.4 Validation results at epoch 15 with identical training (SI-SDRi in dB; mean = average over 2/3/4 speakers) 21
4.5 Size and compute of the deployed graphs 22
4.6 SI-SDRi (dB) of the reduced-precision models (change against FP32 in brackets) 23
4.7 On-device measurements on the Samsung SM-S731B (INT8, 4 CPU threads) 24
4.8 On-device speed for one and four CPU threads (INT8) 24
4.9 Sliding-window streaming on the phone (3 s window, 3-speaker test mixture) 25
4.10 Time per 3 s window in ms (and speed relative to real time) on three smartphones, INT8 25
4.11 Real-time streaming on each phone (3 s window; 0.4 s = 200 + 200 ms, 1 s = 500 + 500 ms latency) 26
4.12 Streaming quality (SI-SDRi, dB) for different latencies (3 s window) 26
4.13 Capabilities of existing systems and the proposed system 27

List of Acronyms

Acronym	Full Form
TSE	Target Speech (Speaker) Extraction
BSS	Blind Source Separation
SI-SDR	Scale-Invariant Signal-to-Distortion Ratio
SI-SDRi	SI-SDR improvement over the mixture
SDR / SIR / SAR	Signal-to-Distortion / -Interference / -Artifact Ratio
SNR	Signal-to-Noise Ratio
PESQ	Perceptual Evaluation of Speech Quality
STOI	Short-Time Objective Intelligibility
NSR	Negative-SI-SDRi Rate (wrong speaker or failed extraction)
FiLM	Feature-wise Linear Modulation
GES	Gated Embedding Split
MGFA	Multi-stage Gated Feature Aggregation
SE	Squeeze-and-Excitation (channel attention)
MHSA	Multi-Head Self-Attention
DIFF	Differential attention
gLN	global Layer Normalisation
DS-conv	Depthwise-Separable convolution
PReLU	Parametric Rectified Linear Unit
CE	Cross-Entropy
ECAPA-TDNN	Emphasized Channel Attention, Propagation and Aggregation TDNN
ONNX	Open Neural Network Exchange
ORT	ONNX Runtime
FP32 / FP16 / INT8	32-bit float / 16-bit float / 8-bit integer number formats
RTF	Real-Time Factor (processing time / audio duration)
LR	Learning Rate
CPU / GPU	Central / Graphics Processing Unit
DNN	Deep Neural Network
ASR	Automatic Speech Recognition
MOS	Mean Opinion Score
STFT	Short-Time Fourier Transform

CHAPTER 1
Introduction

1.1 Background

In everyday environments speech rarely arrives alone. A listener in a restaurant, a meeting room or a family gathering hears several people talking at the same time, together with the target talker. Human listeners can attend to one voice and ignore the others, an ability known as the *cocktail-party effect* [1]. Machines and hearing-impaired listeners find this much harder: hearing aids amplify all talkers equally, and speech recognisers and voice assistants make many more errors when another person speaks over the user.

Two families of deep-learning methods address overlapping speech. *Blind source separation* (BSS) splits a mixture into all of its speakers [7], [9], [10], but it must know how many speakers there are, it returns the sources in an arbitrary order, and its cost grows with the number of outputs. *Target speech extraction* (TSE) instead uses a short enrollment recording of the wanted speaker and returns only that speaker's voice [2]-[6]. TSE answers the question a device actually needs answered ("what is my user saying?") with one output, whatever the number of interfering talkers.

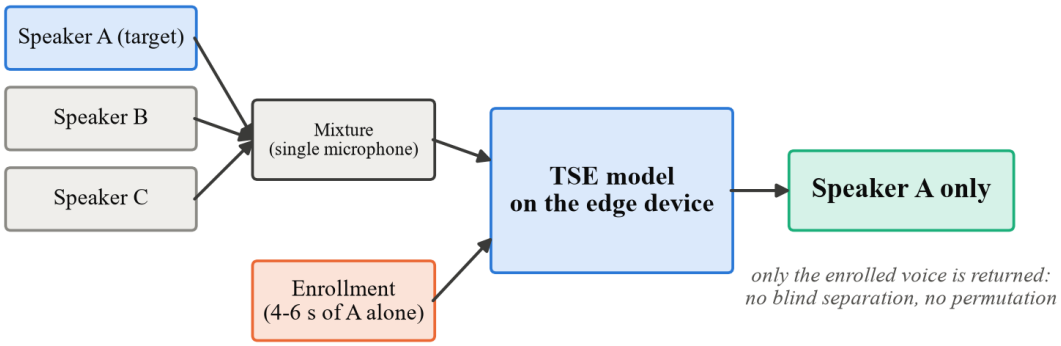


Figure 1.1: Target speech extraction: a short enrollment of speaker A selects which voice is extracted from a single-microphone mixture of several talkers

1.2 Motivation for On-Device Processing

The products that need TSE most are small and run on batteries: hearing aids, earbuds, smart speakers, conference-room units and phones. Running the model on the device, at the edge, rather than in the cloud helps in four ways:

- **Privacy:** speech and the user's enrollment never leave the device, which matters under data protection law such as India's DPDP Act, 2023.

- **Latency:** ITU-T G.114 [36] recommends at most 150 ms of one-way delay for a conversation, and a round trip to a cloud server can use up much of that budget before any processing starts.
- **Availability:** the device keeps working without network coverage and without running costs.
- **Cost:** each device does its own processing, so no server has to be paid for or scaled up as the number of users grows.

The catch is that edge hardware is modest. The phones we tested share 7-11 GB of RAM with every other app, run on a battery, and our runtime uses only their CPU cores. A TSE model for the edge therefore has to be small and cheap per second of audio, tolerate reduced numerical precision, and keep up with a live stream.

1.3 Problem Statement

One microphone records a mixture $y = s_t + \Sigma s_i$ of a target talker and one to three interferers. A separate recording of the target talker alone is also available. From these two inputs we want to recover s_t as faithfully as possible. The model must fit the memory and compute of a phone and keep up with live audio, and nobody tells it in advance how many interferers there are.

1.4 Objectives

1. Build a TSE pipeline that anyone can rerun from scripts - indexing the data, choosing enrollments, training and testing - without a single test speaker leaking into training.
2. Design a time-domain network small enough for a phone (below 15 M parameters) that uses the speaker embedding in every stage.
3. Train one model on mixtures of two to four talkers and test it separately on 2-, 3- and 4-speaker sets.
4. Measure quality with signal metrics (SI-SDR, SNR, SDR, SIR, SAR) and perceptual metrics (PESQ, STOI).
5. Get the model onto a phone through ONNX export, INT8 quantisation and streaming, and check after each step that size, speed and quality are still acceptable.
6. Find out whether multi-head or differential attention at the separator bottleneck helps.

1.5 Contributions

- U-eConv-TSE, a U-shaped convolutional network (13.4 M parameters when deployed) that injects the speaker embedding with FiLM and uses speaker-driven gates to split and merge features across stages.
- Dynamic 2-4 speaker training. With it, one model reaches 9.49 / 8.13 / 7.00 dB SI-SDRi on the 2 / 3 / 4-speaker test sets, against 9.61 / 4.84 / 3.13 dB for the same design trained on pairs only.
- TSE Edge, an offline Android app built around the model. We measured every deployment step (ONNX, INT8, sliding-window streaming) and benchmarked the app on three phones.
- Eight metrics for every test sample, including our own BSS-eval code (checked against mir_eval to within 0.01 dB), and an analysis of which errors remain.
- A controlled comparison of multi-head and differential attention in the separator bottleneck.

1.6 Organisation of the Report

Prior work on separation, extraction, conditioning, attention and on-device inference comes in Chapter 2, which ends with the gaps we target. Chapter 3 covers the model, the data, training, the metrics and the deployment pipeline. Results follow in Chapter 4: quality for two to four speakers, the error analysis, the attention study and the phone measurements. We conclude in Chapter 5.

CHAPTER 2
Literature Survey

2.1 Review of Prior Work

2.1.1 Blind Speech Separation

Deep clustering [9] learned time-frequency embeddings that are clustered into speakers, and permutation-invariant training (PIT) [10] solved the label-ordering problem of training a network with several outputs. Conv-TasNet [7] replaced the short-time Fourier transform by a learned convolutional encoder and decoder and estimated masks with a temporal convolutional network, surpassing ideal time-frequency masks on WSJ0-2mix. Dual-path RNN [12] and SepFormer [11] model long sequences with recurrent and Transformer [15] blocks. SuDoRM-RF [8] showed that U-shaped stacks of depthwise-separable convolutions that successively down-sample and up-sample the signal reach competitive quality at a fraction of the compute, which makes this structure attractive for edge devices. All BSS models need the number of speakers and return outputs in an arbitrary order.

2.1.2 Target Speech Extraction

SpeakerBeam [2] introduced speaker-aware networks in which an auxiliary network turns an enrollment utterance into an adaptation vector that modulates the extraction network. Time-domain SpeakerBeam [3] moved this idea to a Conv-TasNet-style network and added a speaker-identification loss to sharpen speaker discrimination. VoiceFilter [4] conditions a spectrogram-masking network on a d-vector, and VoiceFilter-Lite [5] reduced it to a streaming, quantised model for on-device speech recognition, which is the closest prior work to this project's deployment goal. SpEx+ [6] extracts the target with a fully time-domain multi-scale network and shares the encoder between the mixture and the enrollment.

2.1.3 Speaker Conditioning and Attention

Feature-wise linear modulation (FiLM) [13] conditions a network by predicting a per-channel scale and shift from a conditioning vector, a cheap and effective way to inject the speaker identity. Squeeze-and-excitation (SE) [14] re-weights channels using a global summary of the input. Self-attention [15] relates every time step to every other one; the Differential Transformer [16] computes attention as the difference of two softmax maps, which cancels attention mass that both maps share and has been shown to reduce attention noise in language models.

2.1.4 Speaker Embeddings

x-vectors [17] and ECAPA-TDNN [18] are speaker-verification networks whose utterance-level embeddings are widely used as speaker cues. They are trained on thousands of speakers, but they are not trained to guide an extraction network; this project compares them with a jointly trained speaker encoder.

2.1.5 Datasets and Evaluation

LibriSpeech [19] provides read English speech from over two thousand speakers. LibriMix [20] builds Libri2Mix and Libri3Mix mixtures from it and is the standard open benchmark for separation and extraction. Quality is measured with the scale-invariant SDR [21], the BSS-eval decomposition into distortion, interference and artifacts [22], PESQ [23] and STOI [24].

2.1.6 Efficient On-Device Inference

Integer quantisation [25] stores weights and computes with 8-bit integers, reducing model size four-fold and speeding up inference on processors with integer vector units. ONNX and ONNX Runtime [26] provide a hardware-independent model format and an optimised runtime for CPUs, mobile and embedded devices. VoiceFilter-Lite [5] demonstrated that quantised, streaming TSE is practical on phones.

Table 2.1: Comparison of representative separation and extraction systems

System	Task	Speaker cue	Speakers evaluated	Key property
Conv-TasNet [7]	BSS	-	2-3 (separate models)	Learned encoder, TCN masks
SuDoRM-RF [8]	BSS	-	2 / universal	Efficient U-shaped conv blocks
SpeakerBeam [2]	TSE	adaptation layer	2	Speaker-aware network
TD-SpeakerBeam [3]	TSE	multiplicative	2	Time domain, speaker ID loss
VoiceFilter [4]	TSE	d-vector concat	2	Spectrogram masking
VoiceFilter-Lite [5]	TSE	d-vector	2 (+ noise)	Streaming, quantised, on-device ASR
SpEx+ [6]	TSE	multiplicative	2	Shared encoder, multi-scale
This work	TSE	FiLM + gates	**2, 3 and 4 (one model)**	**U-shaped conv, ONNX + INT8, streaming**

2.2 Gaps Identified

- **Fixed two-speaker evaluation.** Most TSE systems are trained and tested only on two-speaker mixtures, although real scenes often contain more talkers. Few papers show what happens when a third or fourth voice joins: in our tests a two-speaker model fell from 9.61 to 4.84 dB SI-SDRi once a third talker was added.

- **Server-side evaluation.** Model size, quantisation effects and streaming latency are seldom measured together with extraction quality.
- **Target confusion.** If the interferer sounds like the target, for example two male voices, a model can follow the wrong person. Average scores hide this: in our tests about 8% of the outputs were worse than the input mixture.
- **Limited metrics.** Most papers report only the SI-SDR improvement. That one number cannot tell whether a poor output still contains the other talkers or is distorted by the processing, and it says nothing about how the result sounds.

We address these gaps directly. One model is trained and tested for two to four speakers, the whole path from checkpoint to a quantised streaming model on a phone is measured, the wrong-speaker errors are analysed on their own, and every test sample is scored with eight metrics.

CHAPTER 3
Methodology

3.1 System Architecture

3.1.1 Architecture Overview

Two signals go in: the mixture y of T samples and a recording of the target speaker alone. One signal comes out: the estimate $\hat{s}$ of the target speech. U-eConv-TSE, the network in between, is drawn in Figure 3.1. A learned convolutional encoder maps the mixture to a non-negative feature sequence, and a speaker encoder turns the enrollment into a 256-dimensional embedding e. Six separator stages ($R = 6$) then refine the features. Each stage is a U-shaped convolutional block that receives e. After every stage, a gated embedding split keeps the target-specific features, and a gated aggregation merges them across stages. Finally, a mask head and a transposed-convolution decoder turn the merged features back into a waveform. Because everything stays in the time domain, we never compute a Fourier transform or estimate phase.

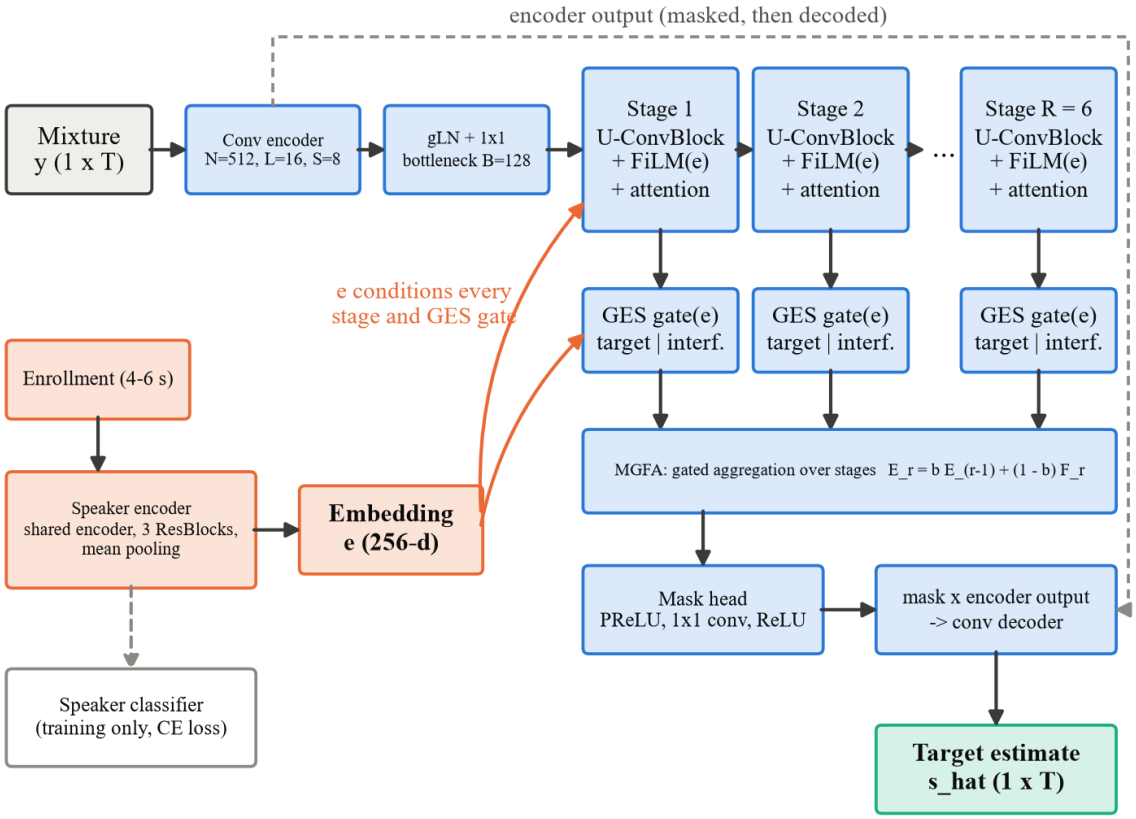


Figure 3.1: Overall architecture of the proposed U-eConv-TSE network (R = 6 stages)

3.1.2 Encoder and Decoder

We use a 1-D convolution with $N = 512$ filters as the encoder. Each filter spans $L = 16$ samples (2 ms at 8 kHz), the stride is $S = 8$, and a ReLU follows, so the encoder emits one 512-dimensional frame per millisecond. Before the separator, global layer normalisation (gLN) and a 1×1 convolution cut these frames down to $B = 128$ channels. On the output side, a transposed convolution mirrors the encoder. It decodes the encoder output after multiplying it by the estimated mask. The network therefore only decides, for each encoder channel and each millisecond, *how much* belongs to the target.

3.1.3 Speaker Encoder

We feed the enrollment through the *same* encoder that processes the mixture. Speaker cue and mixture features then share one learned representation. On top sit a gLN, a 1×1 convolution and three residual blocks, each built from convolutions, batch normalisation, PReLU and max-pooling by 3. Mean pooling over time and a linear layer then produce the 256-dimensional embedding e. During training a linear classifier predicts the speaker identity from e (251 training speakers) and adds a cross-entropy loss, which forces e to be discriminative. The classifier is not used at inference time.

3.1.4 U-ConvBlock Separator with FiLM Conditioning

Each separator stage is a U-ConvBlock (Figure 3.2), adapted from the efficient SuDoRM-RF design [8]. A 1×1 convolution expands the 128 input channels to $H = 512$. Four depthwise-separable convolutions with kernel 5 and stride 2 then halve the time resolution four times (1 ms to 16 ms frames). At the coarsest scale the speaker embedding is injected by FiLM [13]:

$$FiLM(h, e) = \gamma(e) \odot h + \beta(e) \tag{3.1}$$

where γ and β are linear functions of e, initialised so that the layer starts as the identity. An attention module (Section 3.1.6) follows. The block then up-samples back to the input resolution, adding the skip connection of each scale, and a 1×1 convolution and a residual connection return 128 channels. The multi-resolution path lets every stage combine fine detail (pitch harmonics) with long context (syllables and words) at low cost, because most computation happens at reduced time resolution.

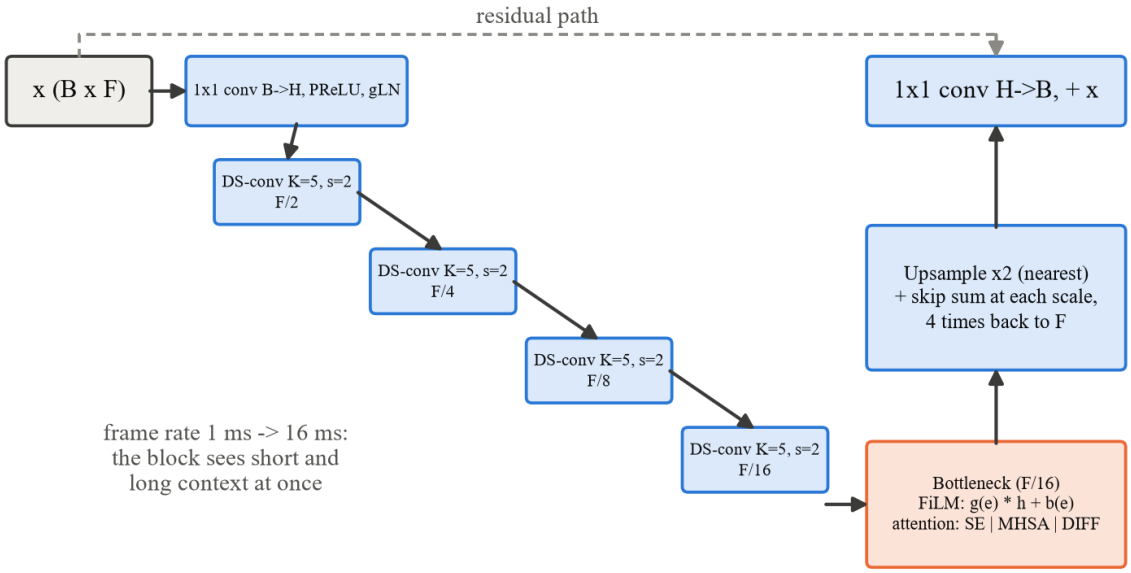


Figure 3.2: One U-ConvBlock separator stage with FiLM conditioning and bottleneck attention

3.1.5 Gated Embedding Split and Multi-stage Gated Feature Aggregation

After every stage a *Gated Embedding Split* (GES) separates the features into a target stream and an interference stream. A 1×1 convolution produces two halves a and g; the gate is conditioned on the speaker, $\mathrm{glu} = a \odot \sigma(g + W e)$, and a second 1×1 convolution with group normalisation splits the result into target features F_t and interference features F_i. The interference stream is decoded only during training, where an auxiliary loss asks it to reconstruct the interfering speech; it is removed for inference. *Multi-stage Gated Feature Aggregation* (MGFA) combines the target features of all stages with a learned per-channel gate:

$$E_r = b \odot E_(r\text{-}1) + (1 - b) \odot F_t,r\,, \quad b = \sigma(w), \quad E_1 = F_t,1 \tag{3.2}$$

so that the mask head sees both early, fine-grained and late, refined features.

3.1.6 Bottleneck Attention: Squeeze-and-Excitation, Multi-Head and Differential

The attention module at the U-block bottleneck is a configurable component (`model.attention`). The baseline is **squeeze-and-excitation (SE)** [14], a channel attention: the features are averaged over time, passed through a two-layer bottleneck MLP and a sigmoid, and the result re-weights the 512 channels. SE gives one weight per channel for the whole segment and cannot relate different time positions.

Two time-attention replacements were implemented. **Multi-head self-attention (MHSA)** [15] treats each 16 ms frame as a token; with layer normalisation (LN), query, key and value projections of width 256 and four heads of width 64,

$$MHSA(h) = h + W_o \cdot concat_k\ softmax(\ Q_k\ K_k^T / \sqrt{d}\ )\ V_k \quad (3.3)$$

Differential attention (DIFF) [16] splits every query and key into two halves and subtracts two attention maps, cancelling attention that both maps assign in common ("attention noise"):

$$DiffAttn(h) = (\ softmax(Q_1K_1^T/\sqrt{d}) - \lambda\ softmax(Q_2K_2^T/\sqrt{d})\ )\ V \quad (3.4)$$

with a learned scalar $\lambda = \exp(\lambda_q1 \cdot \lambda_k1) - \exp(\lambda_q2 \cdot \lambda_k2) + \lambda_init$, where $\lambda_init = 0.8 - 0.6 \cdot \exp(-0.3 \cdot r)$ depends on the stage index r, and a per-head RMS normalisation scaled by $(1 - \lambda_init)$. Two heads of double width are used, so both variants have exactly the same number of parameters. We initialise the output projection of both variants to zero. Each attention block therefore starts as an identity mapping, which keeps early training stable. Differential attention suits extraction well. A mixture contains speech that resembles the target, and attention that reacts to any speech is precisely the common part that the subtraction cancels.

3.2 Dataset and Data Pipeline

3.2.1 Datasets

All audio is 8 kHz, clean (no noise), in the LibriMix *min* mode (mixtures cut to the shortest source). Table 3.1 lists the data. Libri2Mix train-100 provides the 251 training speakers; development and test speakers never appear in training. Libri3Mix test was regenerated exactly from the official LibriMix metadata. Because no public four-speaker set exists, a **Libri4Mix** test set was built by adding a fourth LibriSpeech test-clean speaker (not already present) to every Libri3Mix mixture, at a random level between the quietest and the loudest of the other three, with a fixed random seed.

Table 3.1: Datasets used in this project

Set	Split	Mixtures	TSE samples	Speakers	Use
LibriSpeech train-clean-100	train	dynamic	27,800 per epoch	251	training (dynamic 2-4 spk)
Libri2Mix	dev	3,000	6,000	40	validation (subset 400)
Libri3Mix	dev	3,000	9,000	40	validation (subset 400)
Libri4Mix (ours)	dev	3,000	12,000	40	validation (subset 400)
Libri2Mix	test	3,000	6,000	40	final evaluation
Libri3Mix	test	3,000	9,000	40	final evaluation
Libri4Mix (ours)	test	3,000	12,000	40	final evaluation

Each mixture with n speakers gives n TSE samples, one per possible target speaker.

3.2.2 Enrollment Protocol

LibriMix contains no enrollment recordings, so they are taken from the same speaker's other utterances. For a sample with target speaker S and target utterance U, the enrollment is a clean utterance of S with a different utterance ID, from the same data split. During training a new enrollment is drawn at random every time (4 s crops); for validation and test one enrollment per sample is fixed with a seeded random generator and saved to a table (6 s, centre crop), so every model is evaluated on identical mixture-enrollment pairs.

3.2.3 Dynamic Multi-Speaker Mixing

Instead of the fixed Libri2Mix training mixtures, every training example is mixed on the fly (Figure 3.3). A random 3 s crop of the target utterance is combined with one, two or three interferers drawn from other training speakers, with probabilities 0.40, 0.35 and 0.25 for two, three and four speakers in total. Each source receives an independent level in [−5, +5] dB, and with probability 0.25 an interferer is active for only part of the segment (at least half), which imitates people starting and stopping. The model therefore never sees the same mixture twice and learns to handle a variable number of talkers.

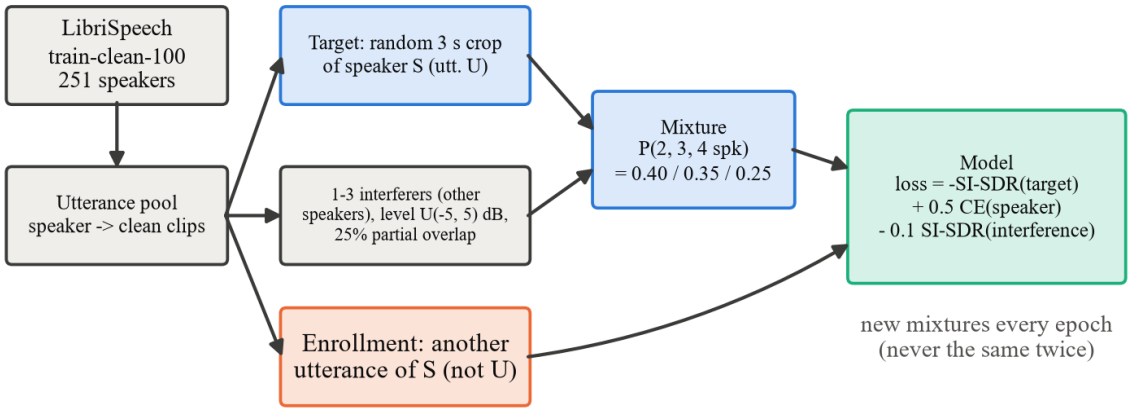


Figure 3.3: Training data pipeline with dynamic 2-4 speaker mixing

3.3 Training Objective and Procedure

The scale-invariant signal-to-distortion ratio (SI-SDR) [21] between the estimate $\hat{s}$ and the reference s is

$$\text{SI-SDR}(\hat{s}, s) = 10 \log_{10}(\|\alpha\, s\|^2 / \|\alpha\, s - \hat{s}\|^2), \quad \alpha = \langle \hat{s}, s \rangle / \|s\|^2 \tag{3.5}$$

after removing the mean of both signals. The training loss combines the negative SI-SDR of the target, the speaker-classification cross-entropy (CE) and the negative SI-SDR of the auxiliary interference estimate $\hat{s}_i$ against the true interference $y - s$:

$$\mathcal{L} = -SI\text{-}SDR(\hat{s}, s) + 0.5 \cdot CE(e) - 0.1 \cdot SI\text{-}SDR(\hat{s}_i, y - s) \quad (3.6)$$

Table 3.2: Model and training hyper-parameters

Parameter	Value
Encoder N / L / S	512 / 16 / 8
Bottleneck B, U-block width H	128, 512
U-block depth Q, kernel K	4, 5
Stages R	6 (proposed), 3 (compact)
Speaker embedding	256-d, ResNet encoder [256, 512, 512]
Segment / enrollment length	3 s / 4 s (train), full / 6 s (test)
Batch size, optimiser	8, Adam [35]
Learning rate	1e-3, halved after 3 epochs without improvement
Epochs	80 (from scratch)
Gradient clipping	5.0
Loss weights	λ_CE = 0.5, λ_int = 0.1
Model selection	best mean validation SI-SDRi over 2/3/4-speaker dev subsets
Training hardware / time	NVIDIA RTX 5090, about 4 min per epoch (about 5.5 h)

Three models are compared in Chapter 4. The **2-speaker model** (R = 3) is trained on the fixed Libri2Mix training mixtures. The **fine-tuned model** continues training the 2-speaker model for 25 epochs with dynamic 2-4 speaker mixing at a learning rate of 2.5e-4. The **proposed model** (R = 6) is trained from scratch for 80 epochs with dynamic 2-4 speaker mixing.

3.4 Evaluation Metrics

We score each of the 27,000 test samples with the metrics below. A trailing *i* means improvement: the metric of the output minus the same metric computed on the unprocessed mixture.

- **SI-SDR / SI-SDRi** [21]: overall extraction quality, insensitive to output level. The main metric.
- **SNR / SNRi**: ordinary signal-to-noise ratio after matching the estimate's energy to the reference. Because the model is trained with a scale-invariant loss its output level is arbitrary, so plain SNR would mostly measure loudness; energy matching keeps leakage and distortion in the error.
- **SDR, SIR, SAR** [22]: BSS-eval decomposition. The estimate is projected (512-tap filters) onto the target alone and onto all sources (each interfering speaker

separately). SIR measures leakage of the other speakers, SAR the artifacts introduced by processing, SDR both.

- **PESQ** [23]: perceived speech quality, ITU-T P.862 narrow-band, on a MOS-like scale (about 1 to 4.5).
- **STOI** [24]: predicted intelligibility, between 0 and 1.
- **NSR** (negative SI-SDRi rate): the share of samples where the output is *worse* than the mixture, which in practice means the wrong speaker was extracted. Also reported: the share below 5 dB.

The available BSS-eval library failed under NumPy 2, so we wrote the projection ourselves with NumPy and SciPy (a Cholesky solve of the block-Toeplitz system). On real three-speaker mixtures it agrees with the reference *mir_eval* code to within 0.01 dB, it reproduces our earlier SDR values (mean difference below 0.001 dB), and it runs about 20 times faster.

3.5 Edge Deployment Pipeline

3.5.1 Graph Split and ONNX Export

A device needs the enrollment only once per target speaker, but it processes the mixture all the time. We therefore split the network into two graphs (Figure 3.4). The **embedder** maps an enrollment to a 256-d embedding; the **extractor** maps a mixture window plus that embedding to the target window. Training-only parts, the speaker classifier and the interference decoder, are dropped. We exported both graphs to ONNX (operator set 17) with a dynamic time axis, so they accept any window length, and compared them with PyTorch on inputs of other lengths than the one used for export.

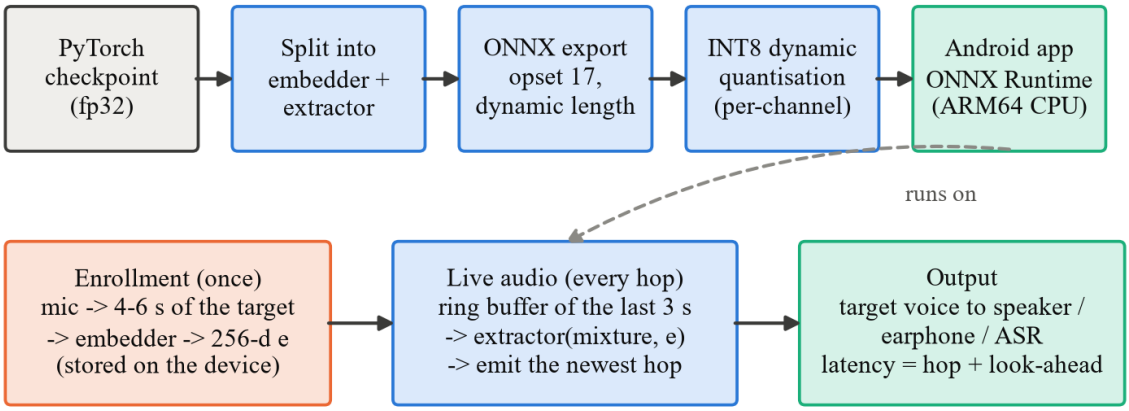


Figure 3.4: Edge deployment pipeline: from the trained checkpoint to streaming inference on the device

3.5.2 Quantisation

We tried two reduced-precision versions. **FP16** stores weights and activations as 16-bit floats. **INT8** uses ONNX Runtime dynamic quantisation: weights become per-channel 8-bit integers, and ONNX Runtime quantises the activations on the fly. INT8 shrinks the R = 6 model files from 51 MB to 14 MB and lets phone CPUs use their integer instructions. Neither version needs retraining or calibration data.

3.5.3 Streaming Inference

The separator is non-causal: global normalisation and two-sided convolutions look at the whole input window. Real-time operation therefore uses a sliding window (Figure 3.5). The device keeps the last $W = 3$ s of audio in a ring buffer; every *hop* it runs the extractor on the buffer and outputs the hop that ends *look-ahead* seconds before the newest sample. The algorithmic latency is hop + look-ahead, and the extractor must process one window in less than one hop of wall-clock time.

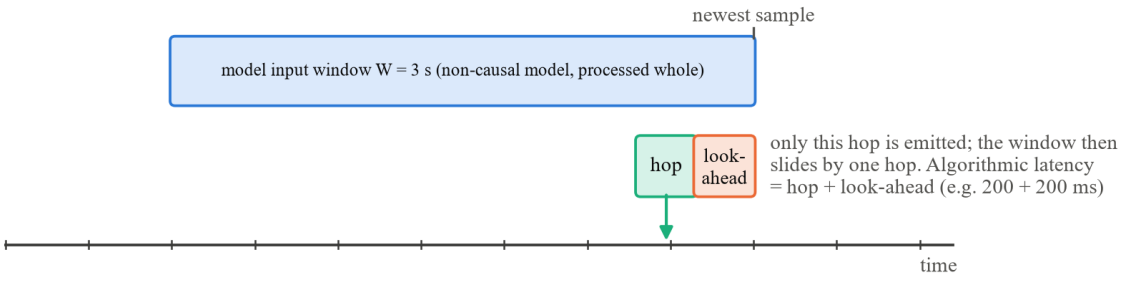


Figure 3.5: Sliding-window streaming: only the newest hop of each 3 s window is output

3.5.4 Target Edge Platform: Android Smartphone Application

The edge platform is an Android smartphone, the most widely available edge device with a microphone, an ARM CPU and a battery. A native Android application, *TSE Edge*, was developed for it (Figure 3.6). The application bundles the INT8 embedder and extractor of both models (R = 3: 8.6 MB, R = 6: 13.6 MB) and the ONNX Runtime Android library (version 1.20, CPU execution provider, ARM64), and it runs the complete extraction on the phone. It requests only the microphone permission and has **no internet permission**, so no audio or result can leave the device; it works in airplane mode.

- **Inference engine** (`TseEngine`, Java). It loads both ONNX graphs from the app package and computes the speaker embedding once per enrollment. Extraction runs on a whole recording or on 3 s windows, with the same sliding-window streaming as our PC code (least-squares level matching, 20 ms crossfade).

- **Audio.** The app records mono audio through Android's `AudioRecord` at 8 kHz. Phones without an 8 kHz mode record at 16, 44.1 or 48 kHz, and the app resamples to 8 kHz behind a 63-tap low-pass filter. Playback goes through `AudioTrack`.
- **User flow.** The user first records 6 s of their own voice. Next they record a conversation with other people talking. The phone then plays back only the enrolled voice.
- **Built-in benchmark.** It times model loading, enrollment and one 3 s window with 1 and 4 CPU threads and reports the real-time factor. It also scores nine bundled test mixtures against the PC result for the same files and tries two streaming latencies. A share button sends everything as text.
- **Verification**: the engine was first run on a desktop JVM with the same ONNX Runtime version and produced exactly the same SI-SDRi as the Python implementation on all bundled test mixtures (difference 0.000 dB).

The detailed measurements in Section 4.3 were made on a Samsung SM-S731B smartphone (Exynos 2400 SoC, 10 cores CPU, 7.0 GB RAM, Android 17); the application was also benchmarked on a vivo I2217 (Snapdragon 8+ Gen 1) and a Nothing A063 (mid-range Snapdragon 778G-class) phone. The application targets Android 7.0 (API 24) and newer on 64-bit and 32-bit ARM phones.

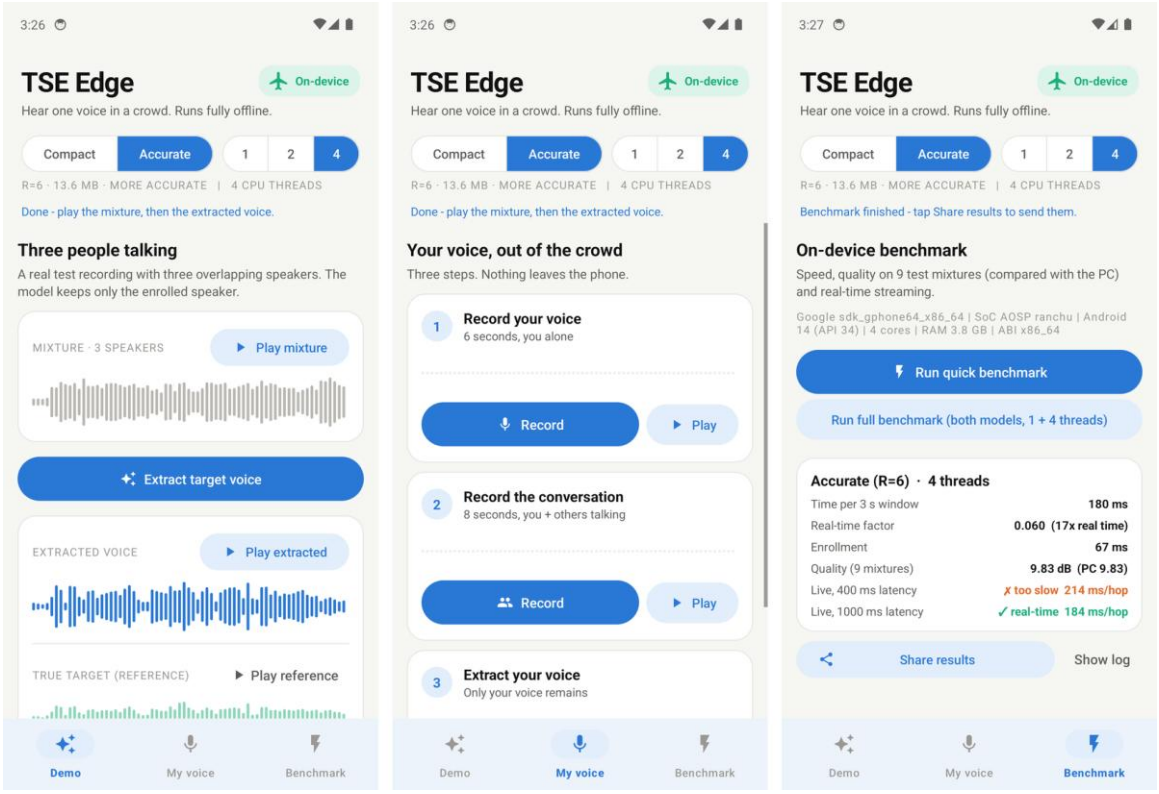


Figure 3.6: The TSE Edge Android application: demo with a 3-speaker test mixture (left), extraction of the user's own voice (centre) and on-device benchmark (right)

3.6 Software and Hardware Setup

Table 3.3: Development environment

Item	Version / specification
Training and evaluation host	Intel Core Ultra 9 285K (24 cores), NVIDIA GeForce RTX 5090 (32 GB)
Operating system	Windows 11
Language / framework	Python 3.13, PyTorch 2.11 (CUDA 12.8)
Export and runtime	ONNX 1.23, ONNX Runtime 1.31 (CPU execution provider)
Metrics	pesq (ITU-T P.862), pystoi, own NumPy/SciPy BSS-eval
Testing	pytest: 81 automated tests (models, data, metrics, training, streaming)
Edge devices	Samsung SM-S731B smartphone (Exynos 2400 SoC, 10 cores CPU, 7.0 GB RAM, Android 17); vivo I2217 (Snapdragon 8+ Gen 1); Nothing A063 (Snapdragon 778G-class); ONNX Runtime Android 1.20 (CPU, ARM64)
Android application	Java, Android API 24-34, built with the Android SDK 34 command-line tools (aapt2, d8, apksigner); APK 26.8 MB including models

CHAPTER 4
Results and Discussion

All numbers in this chapter come from complete runs of the project's evaluation scripts on the full test sets (6,000, 9,000 and 12,000 samples for two, three and four speakers), with the fixed enrollment tables. The edge measurements in Section 4.3 were made on a smartphone by the TSE Edge application itself.

4.1 Results and Comparison

4.1.1 Extraction Quality versus Number of Speakers

Table 4.1: Extraction quality on the 2-, 3- and 4-speaker test sets (higher SI-SDRi / SDRi is better; lower NSR is better)

Test set	Model	SI-SDRi (dB)	median	SDRi (dB)	NSR	< 5 dB
2 speakers	R=3, 2-spk training	9.61	10.98	10.12	5.6%	12.0%
2 speakers	R=3, fine-tuned 2-4 spk	8.73	10.48	9.41	8.8%	17.0%
2 speakers	**R=6, 2-4 spk (proposed)**	9.49	11.11	10.15	7.5%	13.0%
3 speakers	R=3, 2-spk training	4.84	5.36	4.97	7.1%	46.1%
3 speakers	R=3, fine-tuned 2-4 spk	7.19	8.49	7.78	8.3%	22.4%
3 speakers	**R=6, 2-4 spk (proposed)**	8.13	9.60	8.76	7.8%	17.6%
4 speakers *	R=3, 2-spk training	3.13	3.40	3.20	8.9%	76.9%
4 speakers *	R=3, fine-tuned 2-4 spk	6.06	7.05	6.63	8.5%	30.3%
4 speakers *	**R=6, 2-4 spk (proposed)**	7.00	8.33	7.61	8.5%	23.9%

** Libri4Mix is this project's four-speaker extension of Libri3Mix, not an official benchmark.*

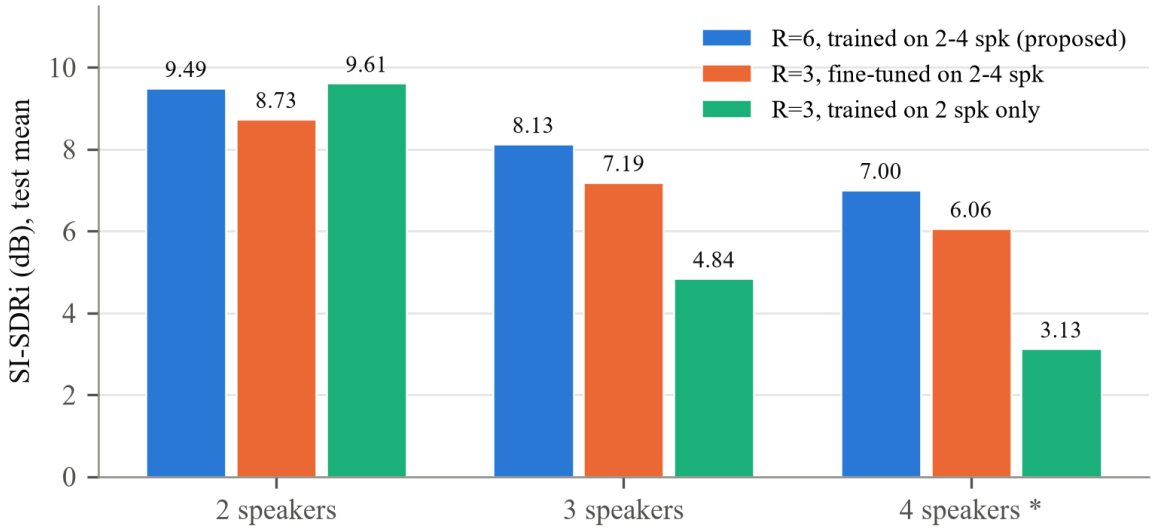


Figure 4.1: Mean SI-SDRi on the test sets as the number of speakers grows

Table 4.1 and Figure 4.1 show the central result. The model trained only on two-speaker mixtures works well on two speakers (9.61 dB) but loses about half of its improvement

when a third speaker is added (4.84 dB) and more with a fourth (3.13 dB); 76.9% of its four-speaker outputs improve by less than 5 dB. Training with dynamic 2-4 speaker mixing removes most of this loss. With our R = 6 model the numbers rise to 8.13 dB on three speakers and 7.00 dB on four, gains of 3.29 and 3.88 dB. On two speakers it stays within 0.12 dB of the specialised model. One model is therefore enough, whether two, three or four people are talking.

Fine-tuning the existing two-speaker model is a cheaper route: it recovers most of the multi-speaker quality in 25 epochs, but it costs 0.9 dB on two speakers and stays about 1 dB behind the proposed model on three and four speakers. Between these two models we changed two things at once: the depth (3 → 6 stages) and the training (25 fine-tuning epochs → 80 epochs from scratch). We therefore cannot credit the gain to depth alone.

4.1.2 Perceptual Quality, Interference and Artifacts

Table 4.2: Signal and perceptual metrics on the test sets (mixture → output for PESQ and STOI)

Test set	Model	SNRi (dB)	SIRi (dB)	SAR (dB)	PESQ	STOI
2 speakers	R=3, 2-spk training	-8.46	17.26	12.32	1.60 → 2.25	0.713 → 0.858
2 speakers	R=3, fine-tuned 2-4 spk	-8.42	16.74	11.44	1.60 → 2.19	0.713 → 0.841
2 speakers	**R=6, 2-4 spk (proposed)**	7.80	17.61	12.22	1.60 → 2.26	0.713 → 0.854
3 speakers	R=3, 2-spk training	-6.19	7.33	8.77	1.37 → 1.58	0.592 → 0.703
3 speakers	R=3, fine-tuned 2-4 spk	-6.36	12.85	8.02	1.37 → 1.67	0.592 → 0.734
3 speakers	**R=6, 2-4 spk (proposed)**	5.58	14.39	8.77	1.37 → 1.75	0.592 → 0.756
4 speakers	R=3, 2-spk training	-5.07	4.69	7.89	1.30 → 1.40	0.528 → 0.602
4 speakers	R=3, fine-tuned 2-4 spk	-5.36	10.49	6.53	1.30 → 1.48	0.528 → 0.653
4 speakers	**R=6, 2-4 spk (proposed)**	4.16	11.93	7.19	1.30 → 1.54	0.528 → 0.679

The BSS-eval decomposition explains *where* the improvement comes from. For the proposed model on three speakers the interference is suppressed by 14.39 dB (SIRi), while the artifacts stay at an SAR of 8.77 dB; the output is therefore limited more by processing artifacts than by leftover speakers. Perceived quality rises from a PESQ of 1.37 for the mixture to 1.75, and predicted intelligibility from 0.592 to 0.756. The two-speaker-only model suppresses the interferers much less on three and four speakers (SIRi 7.33 and 4.69 dB, against 14.39 and 11.93 dB for the proposed model), which confirms that its failure mode is leakage of the extra talkers.

4.1.3 Training Behaviour

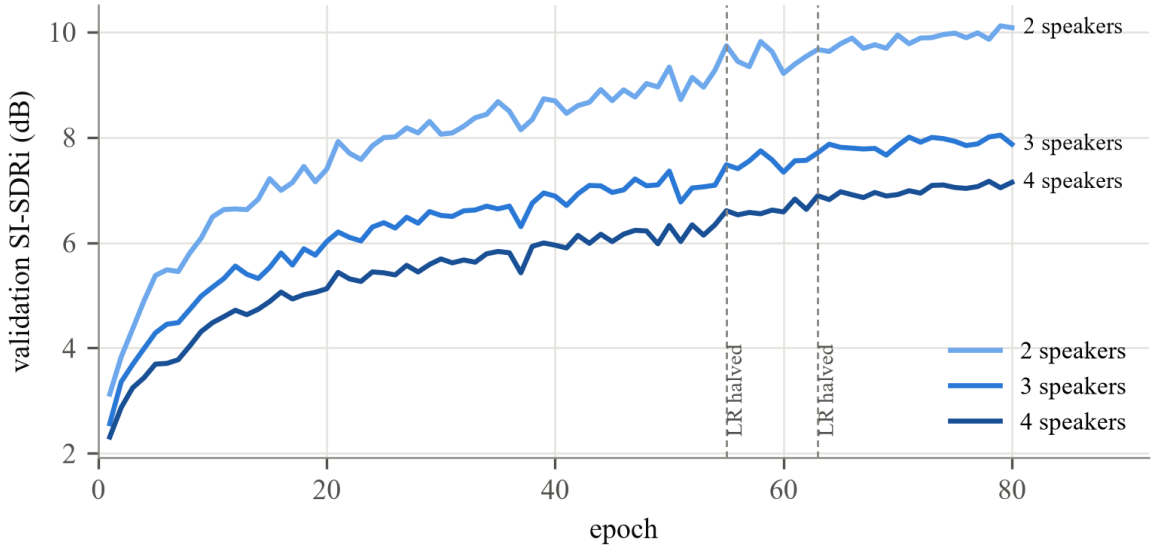


Figure 4.2: Validation SI-SDRi of the proposed model during training, for each number of speakers

Figure 4.2 shows that the three validation curves rise together and stay ordered by difficulty throughout training. The learning rate was halved twice, after which the curves still rise slowly; the best checkpoint was reached at epoch 79 of 80, so the model had not fully converged and a longer schedule would likely improve it further.

4.1.4 Error Analysis

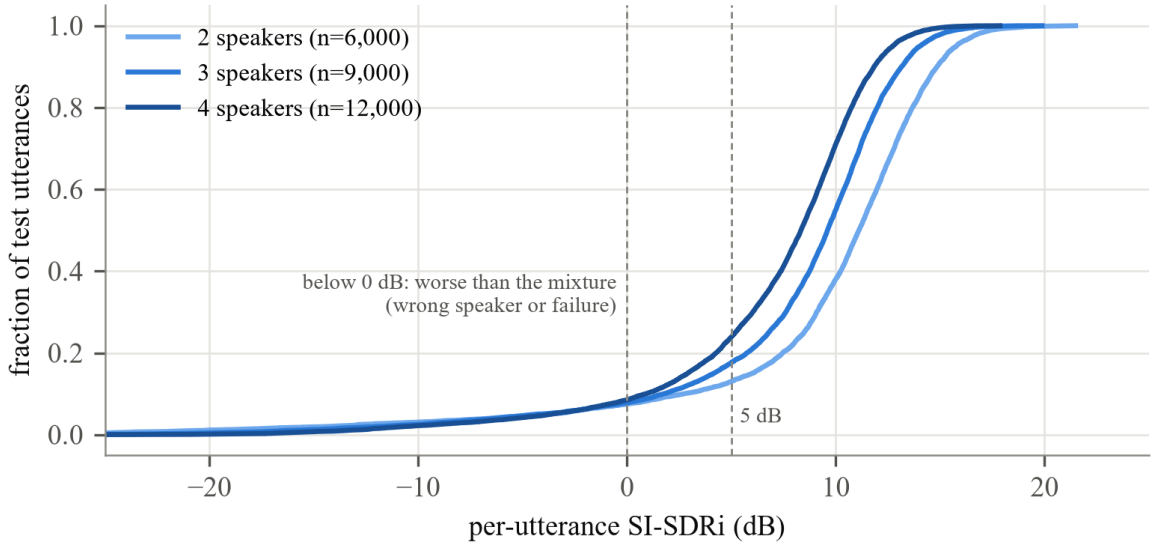


Figure 4.3: Cumulative distribution of per-utterance SI-SDRi of the proposed model

Average scores hide two different kinds of error (Figure 4.3). Most outputs are good: the median SI-SDRi is 11.11, 9.60 and 8.33 dB for two, three and four speakers. A small group of outputs, however, lies below 0 dB (NSR 7.5%, 7.8% and 8.5%): in these cases the model

followed the wrong speaker. Unlike leakage, this error did not decrease with more training data or more stages.

Table 4.3: SI-SDRi (dB) of the proposed model by speaker gender (same gender: at least one interferer has the target's gender)

Test set	same gender	n	different gender	n
2 speakers	8.55	2,972	10.42	3,028
3 speakers	7.65	6,714	9.52	2,286
4 speakers	6.75	10,468	8.73	1,532

Table 4.3 shows that wrong-speaker errors concentrate where voices are similar. When an interferer has the same gender as the target, SI-SDRi is 1.9-2.0 dB lower than when all interferers have the other gender. A finer analysis with a pretrained speaker-verification model (ECAPA-TDNN [18]) measured the similarity of every target-interferer pair: on two speakers the wrong-speaker rate rises from 7.0% for the least similar fifth of pairs to 10.2% for the most similar fifth. The weak point of the system is therefore the speaker cue, not the separation capacity.

4.1.5 Qualitative Example

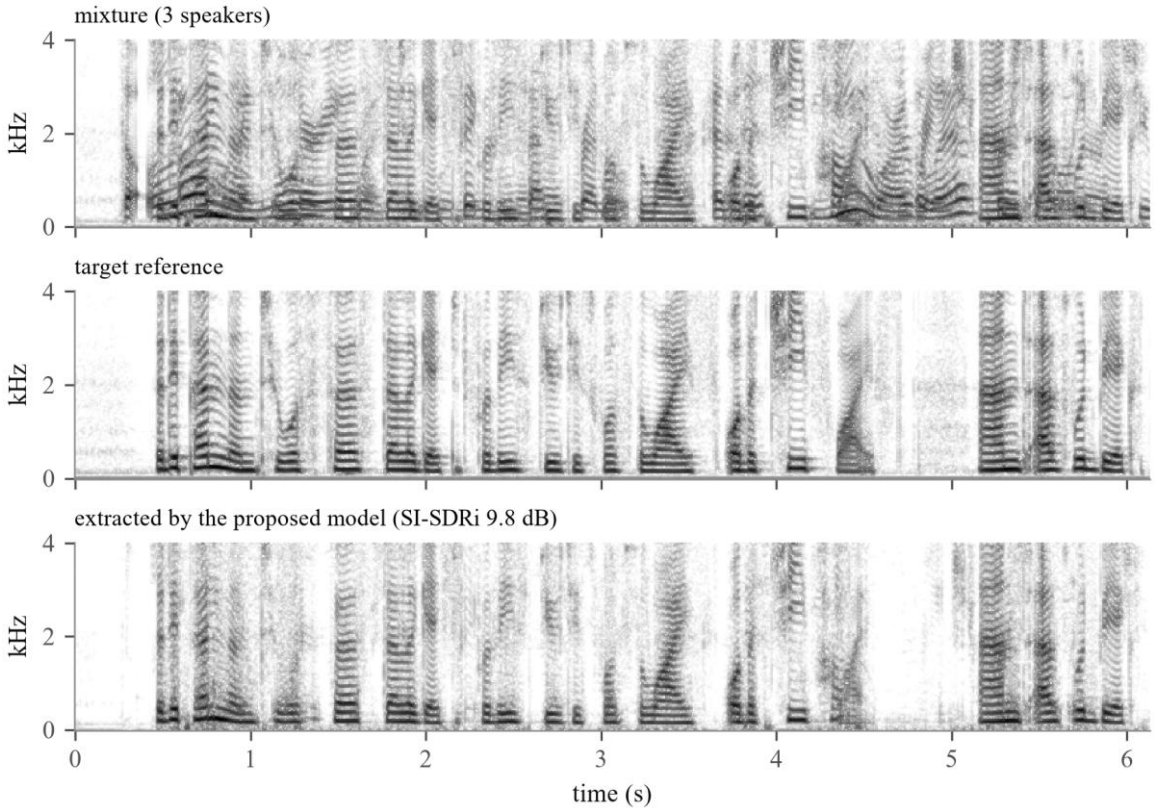


Figure 4.4: Spectrograms of a three-speaker test mixture, the target reference and the extracted speech (an utterance at the median SI-SDRi of the test set)

Figure 4.4 shows a three-speaker test mixture chosen automatically as the utterance whose SI-SDRi is closest to the test median (9.76 dB), so it is representative rather than selected. The harmonics of the two interfering speakers that fill the gaps of the target are removed, while the target's harmonic structure and its silences (for example around 4.6-5.1 s) are preserved.

4.2 Attention Study: Multi-Head and Differential Attention

To test whether attention over time helps the separator, the SE channel attention was replaced by multi-head self-attention (MHSA) or differential attention (DIFF), with everything else identical: same data, recipe, random seed and number of epochs as the proposed model. Only the attention module changes. Both attention variants add 2.76 M parameters (16.36 M against 13.60 M in total) and about 4-7% training time per step.

The two attention runs were stopped after **17 (MHSA) and 15 (DIFF) of the planned 80 epochs** (see below). The comparison is therefore made at matched epochs: all three models are compared at epoch 15, using the per-epoch validation log of the proposed model.

Table 4.4: Validation results at epoch 15 with identical training (SI-SDRi in dB; mean = average over 2/3/4 speakers)

Bottleneck attention	2 spk	3 spk	4 spk	mean	best ≤ ep. 15	NSR
SE (proposed model, channel attention)	7.22	5.53	4.88	5.88	5.88	9.6%
MHSA (multi-head self-attention)	5.07	4.28	3.61	4.32	4.32	8.6%
DIFF (differential attention)	5.47	4.37	3.82	4.55	4.55	10.6%

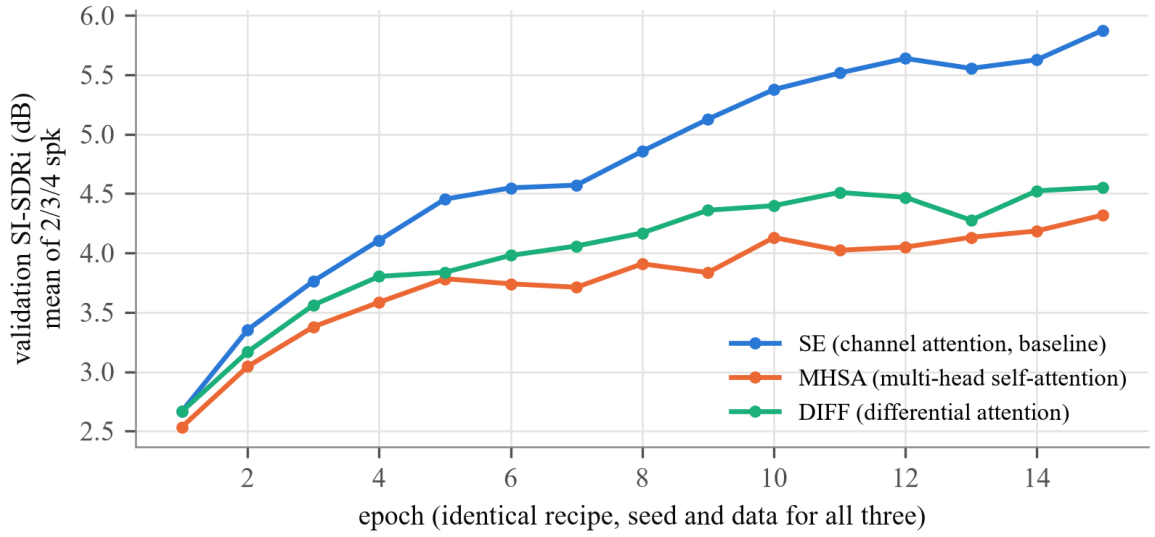


Figure 4.5: Validation SI-SDRi per epoch for SE, MHSA and DIFF bottleneck attention (identical recipe; first 15 epochs)

At epoch 15 the mean validation SI-SDRi is: SE 5.88 dB, DIFF 4.55 dB, MHSA 4.32 dB. SE is ahead by 1.32 dB, far more than the typical epoch-to-epoch change of the validation score (0.21 dB), so the difference is real. Two observations are already useful for the edge goal. First, the attention variants add 20% parameters but little compute, because attention runs at the 16 ms bottleneck resolution where sequences are short. Second, differential attention trains as stably as standard attention, and lags the baseline less than multi-head attention. Because the gap to the SE baseline widened steadily instead of closing (MHSA from -0.13 dB at epoch 1 to -1.59 dB at epoch 17; DIFF from 0.00 dB at epoch 1 to -1.32 dB at epoch 15), and finishing the 80 epochs would have taken about nine more hours, both runs were stopped. With the same recipe and learning rate as the convolutional baseline, replacing SE by time attention therefore did not help. A likely reason is that attention layers usually need a learning-rate warm-up and a lower learning rate than this recipe (1e-3 from the first step) provides; this is left for future work. The checkpoints are kept, so the runs can be resumed.

4.3 Edge Deployment Results

4.3.1 Model Size and Complexity

Table 4.5: Size and compute of the deployed graphs

Model	Embedder params (M)	Extractor params (M)	Extractor GFLOPs per s of audio	Embedder GFLOPs (once)	FP32 size (MB)	INT8 size (MB)
R=3 (compact)	3.55	4.98	2.88	9.15	32.7	8.6
R=6 (proposed)	3.55	9.82	5.46	9.15	51.2	13.6

One multiply-add counts as two FLOPs; the sizes add up the ONNX files of both graphs.

The split pays off. Enrollment costs 9.15 GFLOPs, but only once per speaker. The extractor, which runs all the time, needs 5.46 GFLOPs per second of audio for R = 6 and about half of that for R = 3. INT8 quantisation cuts the stored model to about a quarter of its size, which fits comfortably in the memory of embedded processors. The exported ONNX graphs reproduce the PyTorch outputs: the SI-SDR between the ONNX and the PyTorch output was 53.4 dB for R = 6 and 58.6 dB for R = 3, i.e. numerically identical for audio purposes.

4.3.2 Effect of Quantisation on Quality

Table 4.6: SI-SDRi (dB) of the reduced-precision models (change against FP32 in brackets)

Model	Variant	2 spk	3 spk	4 spk	agreement with FP32 (dB)
R=6	PyTorch FP32 (reference)	11.08 (+0.00)	8.03 (+0.00)	7.23 (+0.00)	-
R=6	PyTorch FP16	fails (overflow)	fails (overflow)	fails (overflow)	-
R=6	ONNX Runtime FP32	11.08 (+0.00)	8.03 (-0.00)	7.23 (-0.00)	58.7
R=6	ONNX Runtime INT8	10.94 (-0.13)	8.05 (+0.02)	7.16 (-0.07)	25.7
R=3	PyTorch FP32 (reference)	9.75 (+0.00)	6.84 (+0.00)	6.49 (+0.00)	-
R=3	PyTorch FP16	fails (overflow)	fails (overflow)	fails (overflow)	-
R=3	ONNX Runtime FP32	9.75 (-0.00)	6.84 (-0.00)	6.49 (+0.00)	60.5
R=3	ONNX Runtime INT8	9.69 (-0.06)	6.75 (-0.09)	6.45 (-0.04)	24.4

Fixed, seeded subsets of 150 test samples per set (same mixtures and enrollments for all variants). Agreement = median SI-SDR between the variant's output and the FP32 output, 3 speakers.

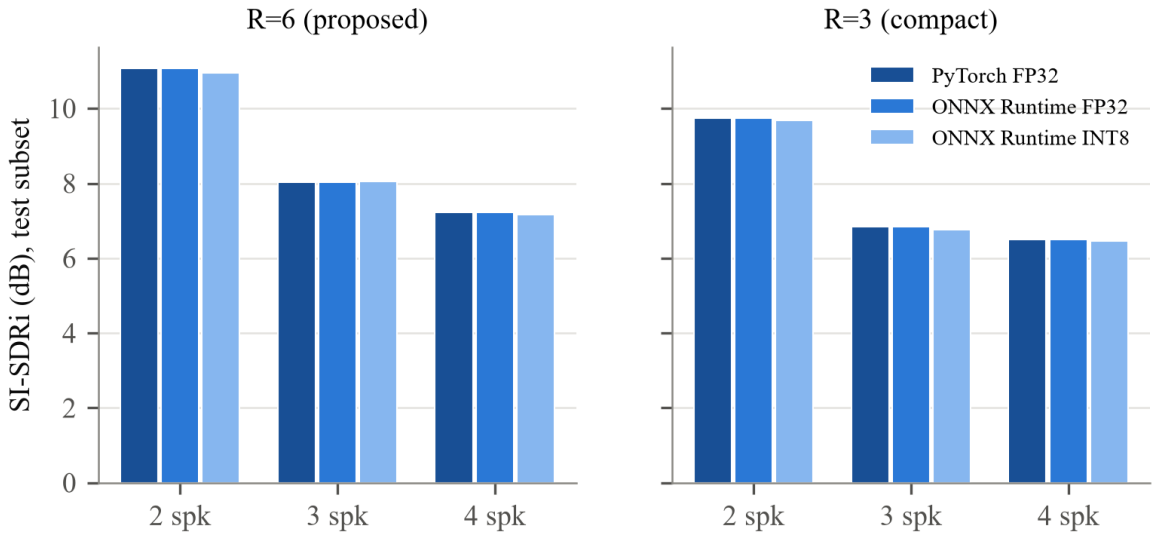


Figure 4.6: SI-SDRi of the FP32 and INT8 versions on 2/3/4-speaker test subsets

The exported ONNX FP32 graph reproduces the PyTorch reference exactly. INT8 quantisation changes SI-SDRi on two, three and four speakers by -0.13, +0.02, -0.07 dB for the proposed R = 6 model; and by -0.06, -0.09, -0.04 dB for the compact R = 3 model, a negligible loss for a four-fold smaller model, which makes INT8 the deployment format. Pure FP16 inference, by contrast, **fails**: the outputs contain non-finite values. To find out why, we traced the FP16 forward pass. The first bad values appear in the gated embedding split (GES). Its group normalisation squares activations that reach about 2×10^4 in the FP32 model, giving roughly 4×10^8, while FP16 cannot hold anything above 65,504. We trained in FP32 with a scale-invariant loss, and nothing in that setup keeps internal levels small. Running in FP16 would need the normalisation layers kept in FP32, or a short precision-

aware fine-tuning. The lesson for deployment: for this model, 8-bit weights with FP32 activations are safer than simply halving the float precision.

4.3.3 Measurements on the Smartphone

The TSE Edge application was installed on a Samsung SM-S731B smartphone (Exynos 2400 SoC, 10 cores CPU, 7.0 GB RAM, Android 17) and run in airplane mode. All numbers below were measured by the application itself on the phone's CPU (ONNX Runtime, INT8, 4 threads) and are reported unchanged.

Table 4.7: On-device measurements on the Samsung SM-S731B (INT8, 4 CPU threads)

Quantity	Accurate model (R=6)	Compact model (R=3)
Model load (both graphs)	292 ms	117 ms
Enrollment, 6 s of speech (once)	80 ms	79 ms
Time per 3 s window (median / p90)	228 / 231 ms	116 / 119 ms
Real-time factor (3 s window)	0.076 (13x faster than real time)	0.039 (26x faster than real time)
Mean SI-SDRi, 9 bundled test mixtures	9.84 dB (PC 9.83 dB)	9.08 dB (PC 9.08 dB)
Largest per-mixture difference to PC	0.17 dB	0.07 dB
Java heap of the application	30 MB	26 MB
Model files in the application	13.6 MB	8.6 MB

Table 4.7 is the main deployment result. The proposed model processes a 3 s window in 228 ms, a real-time factor of 0.076, i.e. about 13 times faster than real time, and the once-per-speaker enrollment takes 80 ms. Extraction quality on the phone (9.84 dB on the nine bundled test mixtures) equals the PC result for the same INT8 model (9.83 dB). The individual mixtures differ by up to 0.17 dB in both directions, most likely because the ARM build of ONNX Runtime uses different integer matrix-multiplication kernels (and activation rounding) than the x86 build; the differences average out.

In interactive use the application extracted the user's voice from 8 s recordings of the phone microphone in 0.61-0.66 s (real-time factor 0.077-0.082), with an enrollment time of 75-140 ms. From the user's point of view the result is available a fraction of a second after the recording ends.

Table 4.8: On-device speed for one and four CPU threads (INT8)

Model	Threads	Load	Enrollment	3 s window	Real-time factor	SI-SDRi (9 mix.)
Compact (R=3)	1	225 ms	104 ms	145 ms	0.048 (21x)	9.08 dB
Compact (R=3)	4	117 ms	79 ms	116 ms	0.039 (26x)	9.08 dB
Accurate (R=6)	1	285 ms	97 ms	275 ms	0.092 (11x)	9.84 dB
Accurate (R=6)	4	292 ms	80 ms	228 ms	0.076 (13x)	9.84 dB

Table 4.8 shows that even a **single CPU core** is enough for both models: one core processes a 3 s window in 145 ms (compact) and 275 ms (accurate). Four threads shorten this by only 20% and 17%, because the model processes one short window at a time and the work per operation is small; on a phone, a single thread is likely the more power-efficient choice. The quality does not depend on the number of threads.

4.3.4 Real-Time Streaming on the Smartphone

Table 4.9: Sliding-window streaming on the phone (3 s window, 3-speaker test mixture)

Model	Thr.	Hop + look-ahead	Latency	Compute / hop	CPU load	Real-time	SI-SDRi
Compact	1	200 + 200 ms	400 ms	142 ms	71%	yes	6.82 dB
Compact	1	500 + 500 ms	1000 ms	142 ms	28%	yes	6.58 dB
Compact	4	200 + 200 ms	400 ms	116 ms	58%	yes	6.82 dB
Compact	4	500 + 500 ms	1000 ms	117 ms	23%	yes	6.58 dB
Accurate	1	200 + 200 ms	400 ms	285 ms	142%	no	7.35 dB
Accurate	1	500 + 500 ms	1000 ms	273 ms	55%	yes	8.07 dB
Accurate	4	200 + 200 ms	400 ms	223 ms	111%	no	7.35 dB
Accurate	4	500 + 500 ms	1000 ms	225 ms	45%	yes	8.07 dB

Real-time: 90% of the hops are computed within the hop duration. CPU load = compute time / hop duration. Whole-utterance SI-SDRi on this mixture: compact 6.76 dB, accurate 8.82 dB.

Table 4.9 shows the streaming behaviour on the phone. Every hop requires one pass over the 3 s window. The **compact model streams in real time with 400 ms of latency even on a single core** (142 ms of computation for every 200 ms of audio, 71% CPU load), with an SI-SDRi of 6.82 dB on this mixture. The accurate model needs about 223 ms per pass, slightly more than a 200 ms hop, so it is run with a 500 ms hop: it then streams in real time with 1 s of latency at 45% CPU load and reaches 8.07 dB (whole-utterance 8.82 dB). The two models therefore offer a clear choice on the device: the compact model for conversation-like latency, the accurate model where one second of delay is acceptable (for example captions, recordings or voice notes). A causal model (Chapter 5) would remove the window overhead and allow lower latency for both.

4.3.5 Results Across Three Smartphones

Table 4.10: Time per 3 s window in ms (and speed relative to real time) on three smartphones, INT8

Phone	SoC	Compact 1 thr.	Compact 4 thr.	Accurate 1 thr.	Accurate 4 thr.	SI-SDRi compact / accurate
Samsung SM-S731B	Exynos 2400 (s5e9945), flagship	145 (21x)	116 (26x)	275 (11x)	228 (13x)	9.08 / 9.84

Phone	SoC	Compact 1 thr.	Compact 4 thr.	Accurate 1 thr.	Accurate 4 thr.	SI-SDRi compact / accurate
vivo I2217	Snapdragon 8+ Gen 1 (SM8475), flagship	192 (16x)	162 (19x)	342 (9x)	307 (10x)	9.08 / 9.84
Nothing A063	Snapdragon 778G-class (SM7325), mid-range	291 (10x)	230 (13x)	534 (6x)	431 (7x)	9.08 / 9.84

Table 4.11: Real-time streaming on each phone (3 s window; 0.4 s = 200 + 200 ms, 1 s = 500 + 500 ms latency)

Phone	Compact 1 thr. 0.4 s	1 s	Compact 4 thr. 0.4 s	1 s	Accurate 1 thr. 0.4 s	1 s	Accurate 4 thr. 0.4 s	1 s
Samsung SM-S731B	yes	yes	yes	yes	no	yes	no	yes
vivo I2217	yes	yes	no	yes	no	yes	no	yes
Nothing A063	no	yes	no	yes	no	no	no	n/r

n/r: not recorded (the log of this phone was cut off before the last result).

We also ran the app on two more phones (Tables 4.10 and 4.11). The extraction quality came out **identical on every phone**: 9.08 dB for the compact and 9.84 dB for the accurate model on the nine bundled mixtures, the same as on the PC. The INT8 models thus behave the same on Samsung Exynos and Qualcomm Snapdragon chips. Speed varies with the hardware, but every phone runs both models many times faster than real time; even the mid-range Nothing A063 handles a 3 s window with the accurate model in 431 ms (7x real time). How much latency a phone can stream in real time depends on its class. On both flagships the compact model streams with 0.4 s of delay on a single core and the accurate model with 1 s, while the mid-range phone needs the 1 s setting even for the compact model. On the vivo I2217 the four-thread runs showed occasional slow hops (up to 1322 ms) while the single-thread runs were steady, which supports running the model on one core: it is more predictable and leaves the other cores free.

4.3.6 Streaming Latency versus Quality (Full Test Subsets)

Table 4.12: Streaming quality (SI-SDRi, dB) for different latencies (3 s window)

Model	Latency	hop / look-ahead (ms)	2 spk	3 spk	4 spk	GPU ms per hop
R=6	100 ms	100 / 0	7.76	6.55	5.52	69.8
R=6	300 ms	200 / 100	8.82	7.49	6.32	70.0
R=6	400 ms	200 / 200	9.18	7.90	6.51	71.0
R=6	500 ms	250 / 250	9.29	8.05	6.60	69.4
R=6	500 ms	200 / 300	9.25	8.06	6.63	70.1
R=6	1000 ms	500 / 500	9.53	8.29	6.88	69.1
R=6	offline	-	9.83	8.63	7.22	-
R=3	100 ms	100 / 0	7.47	6.29	5.26	9.6

Model	Latency	hop / look-ahead (ms)	2 spk	3 spk	4 spk	GPU ms per hop
R=3	300 ms	200 / 100	8.53	6.96	5.78	9.6
R=3	400 ms	200 / 200	8.72	7.30	5.92	9.6
R=3	500 ms	250 / 250	8.77	7.32	5.97	9.4
R=3	500 ms	200 / 300	8.83	7.36	6.00	9.4
R=3	1000 ms	500 / 500	9.00	7.44	6.22	9.5
R=3	offline	-	9.21	7.77	6.39	-

Fixed, seeded subsets of 200 test samples per set; the same samples for every setting.

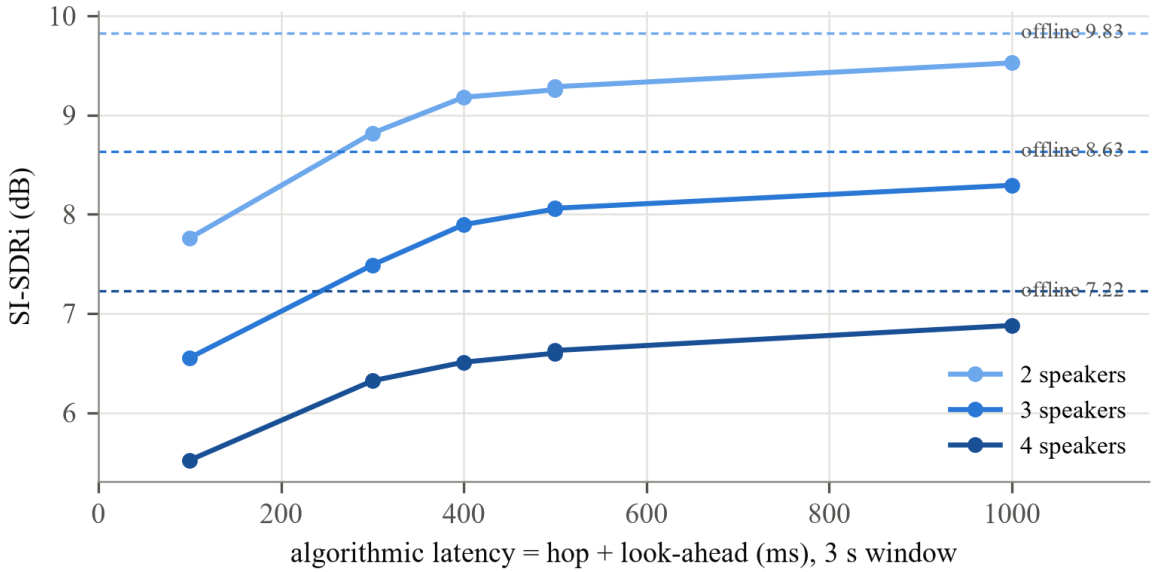


Figure 4.7: Streaming quality of the proposed model against algorithmic latency; dashed lines: whole-utterance (offline) processing

The table extends the single-utterance streaming test of the phone to fixed subsets of the 2-, 3- and 4-speaker test sets (computed on the GPU, so only the quality columns are relevant). The quality loss against whole-utterance processing shrinks as the look-ahead grows, because each emitted hop then has more future context (offline 3-speaker SI-SDRi 8.63 dB).

4.4 Comparison of Existing Systems and the Proposed System

Published TSE systems are trained and evaluated under different conditions (sample rate, data, enrollment selection, number of speakers), so their numbers cannot be compared directly with ours. The following table therefore compares the systems by capability.

Table 4.13: Capabilities of existing systems and the proposed system

Capability	Typical TSE systems [2]-[6]	Proposed system
Number of speakers	trained and tested on 2	one model for 2, 3 and 4
Speaker conditioning	one fusion point	FiLM in every stage + speaker-gated GES

Capability	Typical TSE systems [2]-[6]	Proposed system
Metrics	mostly SI-SDRi	SI-SDRi, SNRi, SDRi, SIRi, SAR, PESQ, STOI, NSR
Error analysis	average scores	wrong-speaker rate, gender and similarity breakdown
Deployment	server GPU (VoiceFilter-Lite [5]: on-device)	ONNX + INT8; offline Android app, real-time on a smartphone CPU
Model size	typically 5-20 M parameters	13.4 M (R=6), 8.5 M (R=3)

4.5 Negative Results

Two attempts to strengthen the speaker cue did not work, and both are reported because they explain the design choices.

- **Pretrained ECAPA-TDNN as the speaker encoder.** Replacing the jointly trained encoder with a frozen ECAPA-TDNN (speaker-verification equal error rate 2.0% against 6.1% for our encoder) made extraction much worse: after 12 epochs the validation SI-SDRi was 2.27 dB with a 29% wrong-speaker rate, against 5.64 dB and 8% for the jointly trained encoder at the same point, and the train-validation gap was 4.4 dB against 1.9 dB. With only 251 training speakers the separator memorised the very distinct fixed embeddings instead of learning to follow an unseen voice. A better speaker-verification embedding is not automatically a better extraction cue.
- **ECAPA-TDNN as a training judge.** Fine-tuning the proposed model with an extra loss that rewards outputs that sound like the target and unlike each interferer (to an ECAPA judge), together with hard training pairs of similar voices, did not beat a control run with the same hard pairs but no judge loss: the judge run reached a best validation SI-SDRi of 8.47 dB when it was stopped after 21 of its 25 epochs, against 8.54 dB for the control, and the control itself scored 9.55 dB on the two-speaker test set against 9.49 dB for the proposed model, i.e. no real gain.

4.6 Limitations

- All results are on clean, read, 8 kHz English speech; noise, reverberation and conversational speech were not evaluated.
- Our four-speaker test set (12,000 samples) is home-made: we built it from Libri3Mix, and no official four-speaker benchmark exists to compare against.

- Because the model looks at future samples, streaming needs a sliding window, and on our phones that meant at least 0.4 s of delay.
- We stopped the attention runs after 15-17 of 80 epochs (Section 4.2), so that comparison covers only early training.
- On-device results come from three phones (two flagship, one mid-range); power and battery consumption were not measured.

CHAPTER 5
Conclusion and Future Work

5.1 Conclusion

We built a target speech extraction system that runs on a phone. It covers the data and training pipeline, a compact speaker-conditioned U-shaped network and an evaluation with eight metrics. It also covers the route onto the device through ONNX, INT8 and sliding-window streaming.

Our main result is that dynamic 2-4 speaker training gives one model for all of these cases. It scores 9.49, 8.13 and 7.00 dB SI-SDRi on two, three and four speakers. The same design trained only on pairs falls to 3.13 dB with four. Most of the gain is better suppression of the extra talkers: on four speakers the SIRi rises from 4.69 to 11.93 dB, and PESQ and STOI rise with it. What remains are mostly wrong-speaker outputs between similar voices. Two ECAPA-based fixes did not help, so a stronger speaker cue is the next thing to work on.

On the deployment side, the embedder/extractor split worked well. The ONNX graphs match PyTorch, and INT8 cuts the model to a quarter of its size for a small loss in quality. Swapping the bottleneck's channel attention for multi-head or differential attention did not help with our training recipe. Both runs fell behind and we stopped them early, although the differential version lagged less. On a Samsung SM-S731B the offline app runs the proposed model 13 times faster than real time and matches the PC quality (9.84 vs 9.83 dB). It streams in real time with 1 s of delay, and the compact model manages 0.4 s on a single core. Target speech extraction on a phone is practical today.

5.2 Future Work

- **Cross-corpus training and testing on TIMIT.** Training jointly on LibriSpeech and TIMIT (with interferers drawn from the same corpus to avoid a recording-condition shortcut) and testing on unseen TIMIT speakers would measure how well the model generalises beyond read audiobooks.
- **Revisit bottleneck attention** with a learning-rate warm-up and a lower learning rate for the attention layers (usual for Transformer blocks), or by fine-tuning the trained model with attention added, and evaluate it on the full test sets.

- **A causal model** trained for streaming (causal convolutions, cumulative normalisation) to bring the latency below 150 ms.
- **A stronger speaker cue** for similar voices, for example contrastive training of the speaker encoder on more speakers (LibriSpeech train-clean-360) or enrollment-mixture cross-attention.
- **Noise and reverberation** (e.g. the noisy LibriMix variants) and real recordings.
- **Quantisation-aware training** and hardware-specific runtimes (e.g. NPU delegates) on the edge device.

Appendix

The complete source code (about 6,000 lines of Python with 81 automated tests) is in the project repository https://github.com/rinolj-12/PJT1-TSE, together with the Android application and all result files. The appendices list the parts most relevant to this report.

Appendix 1: Bottleneck Attention Modules (src/tse/models/layers.py)

```
class MultiHeadAttention(nn.Module):
    """Pre-norm multi-head self-attention over time with a residual path.

    Tokens are the frames of ``x`` (B, C, T); queries/keys/values live in
``attn_dim``
    (< C keeps it cheap). The output projection is zero-initialised, so the
layer is the
    identity at initialisation.
    """

    def __init__(self, channels: int, attn_dim: int = 256, heads: int = 4) -
> None:
        super().__init__()
        if attn_dim % heads:
            raise ValueError(f"attn_dim {attn_dim} not divisible by heads
{heads}")
        self.heads = heads
        self.norm = nn.LayerNorm(channels)
        self.qkv = nn.Linear(channels, 3 * attn_dim)
        self.out = nn.Linear(attn_dim, channels)
        nn.init.zeros_(self.out.weight)
        nn.init.zeros_(self.out.bias)

    def _split(self, t: torch.Tensor) -> torch.Tensor:
        """(B, T, D) -> (B, heads, T, D / heads)."""
        Bn, T, D = t.shape
        return t.view(Bn, T, self.heads, D // self.heads).transpose(1, 2)

    def attend(self, h: torch.Tensor) -> torch.Tensor:
        """(B, T, C) normalised tokens -> (B, heads, T, D / heads) attention
output."""
        q, k, v = (self._split(t) for t in self.qkv(h).chunk(3, dim=-1))
        return F.scaled_dot_product_attention(q, k, v)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """x: (B, C, T) -> (B, C, T)."""
        h = self.norm(x.transpose(1, 2))
        o = self.attend(h)
        o = o.transpose(1, 2).reshape(h.shape[0], h.shape[1], -1)
        return x + self.out(o).transpose(1, 2)



class DifferentialAttention(MultiHeadAttention):
    """Differential attention (Ye et al., 2024, "Differential Transformer")
over time.

    Each head computes two softmax maps from split queries/keys and uses
their difference
    ``(softmax(Q1 K1^T) - lambda * softmax(Q2 K2^T)) V``, which cancels
attention mass both
    maps share (common-mode "noise"). ``lambda`` is re-parameterised as
    ``exp(lq1.lk1) - exp(lq2.lk2) + lambda_init`` with ``lambda_init`` set
by ``depth``; each
```

```
    head output is RMS-normalised and scaled by ``1 - lambda_init``.

    Uses ``heads // 2`` heads of twice the width, so parameters match
    :class:`MultiHeadAttention` with the same ``attn_dim`` and ``heads``.
    """

    def __init__(self, channels: int, attn_dim: int = 256, heads: int = 4,
depth: int = 0) -> None:
        if heads % 2:
            raise ValueError(f"differential attention needs an even number
of heads, got {heads}")
        super().__init__(channels, attn_dim, heads // 2)
        dh = attn_dim // self.heads // 2  # width of each of the two
query/key halves
        self.lambda_init = 0.8 - 0.6 * math.exp(-0.3 * depth)
        self.lq1, self.lk1, self.lq2, self.lk2 = (
            nn.Parameter(torch.randn(dh) * 0.1) for _ in range(4))
        self.head_norm = nn.RMSNorm(2 * dh, eps=1e-5,
elementwise_affine=False)

    def attend(self, h: torch.Tensor) -> torch.Tensor:
        q, k, v = (self._split(t) for t in self.qkv(h).chunk(3, dim=-1))
        q1, q2 = q.chunk(2, dim=-1)
        k1, k2 = k.chunk(2, dim=-1)
        lam = (torch.exp((self.lq1 * self.lk1).sum()) - torch.exp((self.lq2
* self.lk2).sum())
               + self.lambda_init)
        o = (F.scaled_dot_product_attention(q1, k1, v)
             - lam * F.scaled_dot_product_attention(q2, k2, v))
        return self.head_norm(o) * (1.0 - self.lambda_init)
```

Appendix 2: U-ConvBlock Separator Stage (src/tse/models/ublock.py)

```
class UConvBlock(nn.Module):
    """One separator stage: U-shaped multi-resolution conv block with a
residual path.

    ``x (B, Bc, F) -> 1x1 (Bc->H) -> Q stride-2 DS-conv downsamplings ->
fusion(e) +
    attention at F/2^Q -> upsample + skip-sum back to F -> 1x1 (H->Bc) -> +
x``.

    ``attention`` is ``se`` (squeeze-and-excitation channel attention),
``mhsa``
    (multi-head self-attention over time) or ``diff`` (differential
attention over time);
    ``depth`` (stage index) sets the differential-attention ``lambda_init``.

    The time axis is zero-padded to a multiple of ``2**Q`` internally and
cropped back,
    so any length works.
    """

    def __init__(self, B: int, H: int, Q: int, K: int, spk_dim: int, fusion:
str = "film",
                 upsample: str = "nearest", se_reduction: int = 8,
attention: str = "se",
                 attn_dim: int = 256, attn_heads: int = 4, depth: int = 0) -
> None:
        super().__init__()
        self.Q = Q
        self.proj_in = nn.Sequential(nn.Conv1d(B, H, 1), nn.PReLU(),
GlobalLayerNorm(H))
```

```
        self.down = nn.ModuleList([DSConvBlock(H, H, K, stride=2) for _ in
range(Q)])
        self.fusion = make_fusion(fusion, spk_dim, H)
        self.attn = make_attention(attention, H, se_reduction, attn_dim,
attn_heads, depth)
        self.upsample = upsample
        if upsample == "transposed":
            self.up = nn.ModuleList([nn.ConvTranspose1d(H, H, 4, stride=2,
padding=1, groups=H)
                                     for _ in range(Q)])
        self.proj_out = nn.Conv1d(H, B, 1)

    def _up(self, y: torch.Tensor, q: int, size: int) -> torch.Tensor:
        if self.upsample == "transposed":
            y = self.up[q](y)
            return y[..., :size]
        return F.interpolate(y, size=size, mode="nearest")

    def forward(self, x: torch.Tensor, e: torch.Tensor | None) ->
torch.Tensor:
        """x: (B, Bc, F); e: (B, D) or None (no conditioning in this
stage)."""
        T = x.shape[-1]
        mult = 2 ** self.Q
        pad = (mult - T % mult) % mult
        y = self.proj_in(F.pad(x, (0, pad)))
        skips = [y]
        for d in self.down:
            y = d(y)
            skips.append(y)
        if e is not None:
            y = self.fusion(y, e)
        y = self.attn(y)
        for q in reversed(range(self.Q)):
            y = self._up(y, q, skips[q].shape[-1]) + skips[q]
        return self.proj_out(y)[..., :T] + x
```

Appendix 3: BSS-eval SIR / SAR Implementation (src/tse/metrics.py)

```
def _xcorr(a: np.ndarray, b: np.ndarray, n_lags: int, nfft: int) ->
np.ndarray:
    """``r[t] = sum_n a[n] b[n + t]`` for lags ``t = 0 .. n_lags-1`` (rows
of a, b broadcast)."""
    fa, fb = np.fft.rfft(a, nfft), np.fft.rfft(b, nfft)
    return np.fft.irfft(np.conj(fa) * fb, nfft)[..., :n_lags]



def bss_eval_target(est: np.ndarray, ref: np.ndarray, interferers:
np.ndarray,
                    filter_length: int = 512, load_diag: float = 1e-10
                    ) -> tuple[float, float, float]:
    """BSS-eval SDR, SIR and SAR of ``est`` for target ``ref`` (Vincent et
al., 2006; the
    ``bss_eval_sources`` definition of mir_eval / BSS Eval v3 with 512-tap
filters).

    ``interferers`` (K, T) holds each interfering source separately. The
estimate is
    projected (least squares, time-invariant FIR filters) onto the target
alone and onto
    all references: ``s_target = P_target est``, ``e_interf = P_all est -
s_target``,
    ``e_artif = est - P_all est``; ``SDR = |s_target|^2 / |e_interf +
e_artif|^2``,
```

```
    ``SIR = |s_target|^2 / |e_interf|^2`` and ``SAR = |s_target +
e_interf|^2 / |e_artif|^2``.
    SDR depends only on the target projection, so it equals fast_bss_eval's
single-reference
    ``sdr`` (used for the SDR numbers reported before SIR/SAR were added).

    Implemented directly in numpy/scipy (Cholesky solve of the block-
Toeplitz Gram matrix):
    fast_bss_eval's numpy backend fails on numpy>=2 with several references
and its torch
    backend hits an MKL ``lu_solve`` error on some inputs.
    """
    L = filter_length
    refs = np.concatenate([ref[None], interferers]).astype(np.float64)
    x = est.astype(np.float64)
    n, T = refs.shape
    nfft = int(2 ** np.ceil(np.log2(T + L)))
    # Gram matrix G[(i,a),(j,b)] = sum_t r_i[t-a] r_j[t-b] = R_ij[a-b]
(zero-padded signals)
    G = np.empty((n * L, n * L))
    for i in range(n):
        for j in range(i, n):
            r_ij = _xcorr(refs[i], refs[j], L, nfft)  # sum r_i[t]
r_j[t+lag], lag >= 0
            r_ji = _xcorr(refs[j], refs[i], L, nfft)
            lag = np.arange(L)[:, None] - np.arange(L)[None, :]  # a - b
            blk = np.where(lag >= 0, r_ij[np.abs(lag)], r_ji[np.abs(lag)])
            G[i * L:(i + 1) * L, j * L:(j + 1) * L] = blk
            G[j * L:(j + 1) * L, i * L:(i + 1) * L] = blk.T
    # D[(i,a)] = sum_t r_i[t-a] x[t]
    D = _xcorr(refs, x[None], L, nfft).reshape(-1)
    reg = load_diag * np.trace(G) / len(G) + 1e-12
    G[np.diag_indices_from(G)] += reg

    def project(k: int) -> np.ndarray:
        c = scipy.linalg.cho_solve(scipy.linalg.cho_factor(G[:k * L, :k *
L]), D[:k * L])
        return sum(scipy.signal.fftconvolve(refs[i], c[i * L:(i + 1) *
L])[:T] for i in range(k))

    s_target = project(1)
    p_all = project(n)
    e_interf = p_all - s_target
    e_artif = x - p_all
    eps = 1e-20
    sir = 10 * np.log10((s_target @ s_target + eps) / (e_interf @ e_interf +
eps))
    sar = 10 * np.log10((p_all @ p_all + eps) / (e_artif @ e_artif + eps))
    e_total = x - s_target
    sdr = 10 * np.log10((s_target @ s_target + eps) / (e_total @ e_total +
eps))
    return float(sdr), float(sir), float(sar)
```

Appendix 4: On-Device Inference Engine of the Android Application (TseEngine.java)

```
    public TseEngine(byte[] embedderModel, byte[] extractorModel, int
threads) throws OrtException {
        long t0 = System.nanoTime();
        this.threads = threads;
        OrtSession.SessionOptions so = new OrtSession.SessionOptions();
        so.setIntraOpNumThreads(threads);
        so.setInterOpNumThreads(1);
        so.setOptimizationLevel(OrtSession.SessionOptions.OptLevel.ALL_OPT);
```

```
        embedder = env.createSession(embedderModel, so);
        extractor = env.createSession(extractorModel, so);
        loadMillis = (System.nanoTime() - t0) / 1_000_000;
    }

    /** Speaker embedding (256 floats) of an enrollment recording. */
    public float[] embed(float[] enrollment) throws OrtException {
        try (OnnxTensor x = tensor(enrollment);
             OrtSession.Result r =
embedder.run(Collections.singletonMap("enrollment", x))) {
            return ((float[][]) r.get(0).getValue())[0];
        }
    }

    /** Target estimate for a mixture (same length; arbitrary output level,
see {@link #matchLevel}). */
    public float[] extract(float[] mixture, float[] embedding) throws
OrtException {
        try (OnnxTensor x = tensor(mixture); OnnxTensor e =
tensor(embedding)) {
            java.util.HashMap<String, OnnxTensor> in = new
java.util.HashMap<>();  // Map.of needs Android 11+
            in.put("mixture", x);
            in.put("embedding", e);
            try (OrtSession.Result r = extractor.run(in)) {
                return ((float[][]) r.get(0).getValue())[0];
            }
        }
    }
```

Appendix 5: Reproducing the Results

```
# environment
pip install torch torchaudio --index-url
https://download.pytorch.org/whl/cu128
pip install -r requirements.txt pesq onnx onnxruntime
# training (proposed model; attention variants)
python scripts/train.py --config configs/multispk_deep.yaml
powershell -File experiments/attention/run.ps1 diff
# evaluation on 2/3/4 speakers with all metrics
powershell -File experiments/report_eval/run.ps1
# edge: export, quantise, accuracy and CPU latency
python scripts/edge_benchmark.py --models
r6=saved_models/multispk_r6_ep79/best.pt r3=runs/finetune_multispk/best.pt
python scripts/stream_eval.py --model r6_multispk --sets 2spk 3spk 4spk --n
200
# Android application (offline, models inside the APK)
python phone_demo/prepare_phone_assets.py; python
android_app/prepare_assets.py
bash android_app/build_apk.sh          # -> android_app/build/TSE-Edge.apk
# figures and this report
python report/diagrams.py; python report/figures.py; python
report/build_report.py
```

Bibliography

[1] E. C. Cherry, “Some experiments on the recognition of speech, with one and with two ears,” *J. Acoust. Soc. Am.*, vol. 25, no. 5, pp. 975–979, 1953.

[2] K. Žmolíková, M. Delcroix, K. Kinoshita, T. Ochiai, T. Nakatani, L. Burget, and J. Černocký, “SpeakerBeam: Speaker aware neural network for target speaker extraction in speech mixtures,” *IEEE J. Sel. Topics Signal Process.*, vol. 13, no. 4, pp. 800–814, 2019.

[3] M. Delcroix, T. Ochiai, K. Žmolíková, K. Kinoshita, N. Tawara, T. Nakatani, and S. Araki, “Improving speaker discrimination of target speech extraction with time-domain SpeakerBeam,” in *Proc. IEEE ICASSP*, 2020.

[4] Q. Wang, H. Muckenhirn, K. Wilson, P. Sridhar, Z. Wu, J. Hershey, R. A. Saurous, R. J. Weiss, Y. Jia, and I. L. Moreno, “VoiceFilter: Targeted voice separation by speaker-conditioned spectrogram masking,” in *Proc. Interspeech*, 2019.

[5] Q. Wang, I. L. Moreno, M. Saglam, K. Wilson, A. Chiao, R. Liu, Y. He, W. Li, J. Pelecanos, M. Nika, and A. Gruenstein, “VoiceFilter-Lite: Streaming targeted voice separation for on-device speech recognition,” in *Proc. Interspeech*, 2020.

[6] M. Ge, C. Xu, L. Wang, E. S. Chng, J. Dang, and H. Li, “SpEx+: A complete time domain speaker extraction network,” in *Proc. Interspeech*, 2020.

[7] Y. Luo and N. Mesgarani, “Conv-TasNet: Surpassing ideal time–frequency magnitude masking for speech separation,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 27, no. 8, pp. 1256–1266, 2019.

[8] E. Tzinis, Z. Wang, and P. Smaragdis, “Sudo rm -rf: Efficient networks for universal audio source separation,” in *Proc. IEEE Int. Workshop on Machine Learning for Signal Processing (MLSP)*, 2020.

[9] J. R. Hershey, Z. Chen, J. Le Roux, and S. Watanabe, “Deep clustering: Discriminative embeddings for segmentation and separation,” in *Proc. IEEE ICASSP*, 2016, pp. 31–35.

[10] M. Kolbæk, D. Yu, Z.-H. Tan, and J. Jensen, “Multitalker speech separation with utterance-level permutation invariant training of deep recurrent neural networks,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 25, no. 10, pp. 1901–1913, 2017.

[11] C. Subakan, M. Ravanelli, S. Cornell, M. Bronzi, and J. Zhong, “Attention is all you need in speech separation,” in *Proc. IEEE ICASSP*, 2021.

[12] Y. Luo, Z. Chen, and T. Yoshioka, “Dual-path RNN: Efficient long sequence modeling for time-domain single-channel speech separation,” in *Proc. IEEE ICASSP*, 2020.

[13] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” in *Proc. AAAI Conf. Artificial Intelligence*, 2018.

[14] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” in *Proc. IEEE/CVF CVPR*, 2018, pp. 7132–7141.

[15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proc. NeurIPS*, 2017.

[16] T. Ye, L. Dong, Y. Xia, Y. Sun, Y. Zhu, G. Huang, and F. Wei, “Differential Transformer,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025, arXiv:2410.05258.

[17] D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, and S. Khudanpur, "X-vectors: Robust DNN embeddings for speaker recognition," in *Proc. IEEE ICASSP*, 2018, pp. 5329–5333.

[18] B. Desplanques, J. Thienpondt, and K. Demuynck, "ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification," in *Proc. Interspeech*, 2020.

[19] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "LibriSpeech: An ASR corpus based on public domain audio books," in *Proc. IEEE ICASSP*, 2015, pp. 5206–5210.

[20] J. Cosentino, M. Pariente, S. Cornell, A. Deleforge, and E. Vincent, "LibriMix: An open-source dataset for generalizable speech separation," arXiv:2005.11262, 2020.

[21] J. Le Roux, S. Wisdom, H. Erdogan, and J. R. Hershey, "SDR – half-baked or well done?" in *Proc. IEEE ICASSP*, 2019, pp. 626–630.

[22] E. Vincent, R. Gribonval, and C. Févotte, "Performance measurement in blind audio source separation," *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 14, no. 4, pp. 1462–1469, 2006.

[23] A. W. Rix, J. G. Beerends, M. P. Hollier, and A. P. Hekstra, "Perceptual evaluation of speech quality (PESQ) – a new method for speech quality assessment of telephone networks and codecs," in *Proc. IEEE ICASSP*, 2001.

[24] C. H. Taal, R. C. Hendriks, R. Heusdens, and J. Jensen, "An algorithm for intelligibility prediction of time–frequency weighted noisy speech," *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 19, no. 7, pp. 2125–2136, 2011.

[25] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF CVPR*, 2018, pp. 2704–2713.

[26] ONNX Runtime developers, "ONNX Runtime: cross-platform, high performance ML inferencing and training accelerator," 2021. [Online]. Available: https://onnxruntime.ai

[27] ITU-T Recommendation P.862, "Perceptual evaluation of speech quality (PESQ): An objective method for end-to-end speech quality assessment of narrow-band telephone networks and speech codecs," International Telecommunication Union, 2001.

[28] ITU-T Recommendation G.711, "Pulse code modulation (PCM) of voice frequencies," International Telecommunication Union, 1988.

[29] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019.

[30] J. S. Garofolo, L. F. Lamel, W. M. Fisher, J. G. Fiscus, D. S. Pallett, N. L. Dahlgren, and V. Zue, "TIMIT acoustic-phonetic continuous speech corpus," Linguistic Data Consortium, LDC93S1, 1993.

[31] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015, pp. 234–241.

[32] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[33] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, "Language modeling with gated convolutional networks," in *Proc. ICML*, 2017.

[34] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” arXiv:1607.06450, 2016.

[35] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.

[36] ITU-T Recommendation G.114, “One-way transmission time,” International Telecommunication Union, 2003.

Biodata

[Photo – insert here]

Name: Johaan Sam Sanil (23BLC1286)
Mobile No.: **[TBD]**
E-mail: **[TBD]**
Permanent Address: **[TBD]**

[Photo – insert here]

Name: Rinol Jeo Manoj (23BLC1207)
Mobile No.: **[TBD]**
E-mail: **[TBD]**
Permanent Address: **[TBD]**

[Photo – insert here]

Name: Navneeth Regikumar (23BLC1276)
Mobile No.: **[TBD]**
E-mail: **[TBD]**
Permanent Address: **[TBD]**